

INFORME FINAL — ERSO 2026-005

- [INFORME FINAL — ERSO 2026-005](#)
 - [Publicación del DID Document bajo `did:web`](#)
 - [Cómo leer este informe](#)
 - [Contenido](#)
 - [Resumen ejecutivo](#)
- [PARTE I — MARCO CONCEPTUAL: los términos que necesitas](#)
 - [1. La historia en cinco minutos](#)
 - [2. Diccionario de términos](#)
 - [3. Las siete ideas madre](#)
 - [4. El método de trabajo: cómo se piensa un criterio](#)
- [PARTE II — QUÉ PIDE LA ERSO 005, EXPLICADO FRASE POR FRASE](#)
 - [5. La ficha del documento](#)
 - [6. Capacidades del proceso](#)
 - [7. Condiciones del proceso](#)
 - [8. Descripción del proceso](#)
 - [9. Qué debe hacer: los cinco pasos](#)
 - [10. Los cuatro criterios de aceptación](#)
- [PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO FUNCIONA](#)
 - [11. Visión general](#)
 - [12. El entorno para esta ERSO](#)
 - [13. Cómo funciona por dentro \(lo mínimo para entender las pruebas\)](#)
- [PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA PASO Y CADA CRITERIO](#)
 - [14. Cómo funcionan los laboratorios](#)
 - [15. Preparación común: un identificador de prueba único](#)
 - [16. Laboratorio A — Paso 1: generar el `did:web`](#)
 - [17. Laboratorio B — Paso 2: construir el documento](#)
 - [18. CRITERIO 1 — El documento se sirve por HTTPS en la URL calculada](#)
 - [19. CRITERIO 2 — El documento contiene la clave pública correcta](#)
 - [20. CRITERIO 3 — El documento no expone claves privadas ni datos civiles](#)
 - [21. CRITERIO 4 — Un `id` desajustado se rechaza y el camino base opera con la extensión desactivada](#)
 - [22. Pruebas automáticas y recorrido de extremo a extremo](#)
- [PARTE V — PREGUNTAS Y RESPUESTAS](#)
 - [23. Preguntas sobre los conceptos](#)
 - [24. Preguntas por laboratorio y por criterio](#)
- [PARTE VI — CÓMO RESPONDER A CADA CRITERIO](#)
 - [25. Redacción modelo de las cuatro respuestas](#)
 - [26. Lista de comprobación para quien acepta](#)
- [PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN](#)
 - [27. Lo que este informe NO demuestra, y lo que se encontró](#)
 - [28. Si algo no sale como en el informe](#)
 - [29. Operación diaria](#)
- [ANEXOS](#)
 - [Anexo A — Los scripts](#)
 - [Anexo B — Autocomprobación \(con respuestas\)](#)
 - [Anexo C — Ejercicios del método \(con respuesta modelo\)](#)

INFORME FINAL — ERSO 2026-005

Publicación del DID Document bajo `did:web`

Manual de comprensión, construcción y comprobación

Proyecto	Identidad Digital Soberana SSI
ERSo	2026-005 — Publicación del DID Document bajo <code>did:web</code>
Desarrollador asignado (según la ERSo)	Geovani Rincón
Plantilla de pruebas y pruebas funcionales (según la ERSo)	Luis González
Fechas de la ERSo	Elaboración: viernes 18-sep-2026 · Desarrollo: miércoles 30-sep-2026, 7:30 a. m. · Prueba: viernes 2-oct-2026, 7:30 a. m.
Dominio del laboratorio	<code>civica-desarrollo.avance.org.co</code>
Ubicación del proyecto	<code>/home/geovani/Descargas/generic/bitacora/new_proyect</code>
Salidas de terminal de este informe	Obtenidas ejecutando los comandos reales sobre el proyecto, el 1 de octubre de 2026
Complementos	<code>scripts/lab-005-completo.sh</code> (todos los laboratorios en un script) · <code>informes/ERSo-2026-005.md</code> (versión corta) · <code>informes/INFORME-FINAL-ERSo-2026-004.md</code> (el registro sobre el que corre esta ERSo)

Cómo leer este informe

Este documento está escrito para que **cualquier persona** pueda entender qué se pidió, qué se construyó y cómo se comprobó, **sin necesidad de ser experta** en identidad digital o criptografía. Cada término técnico se explica la primera vez que aparece, y toda la Parte I es un diccionario ilustrado de esos términos.

Símbolos que verás:

Símbolo	Significa
	Idea clave: lo más importante de la sección, en una frase
	Qué vas a ver: lo que debes esperar <i>antes</i> de ejecutar un comando
	Qué significa: cómo interpretar lo que el sistema respondió
	Cuidado: una trampa frecuente o un límite que conviene conocer
	Predice antes de ejecutar: una pregunta para que pienses antes de ver el resultado

Cómo leerlo según tu necesidad:

Si eres...	Lee esto
Quien decide o aprueba	El Resumen ejecutivo y la Parte VII (límites y hallazgos)
Quien necesita entender el problema	Parte I (conceptos) y Parte II (qué pide la ERSO)
Quien debe reproducir y comprobar	Partes III y IV (montaje y laboratorios)
Quien prepara la prueba funcional del 2 de octubre	Partes IV, VI y la lista de comprobación (§26)
Quien prepara una exposición o defensa	Partes V y VI (preguntas, respuestas y redacción de los criterios)

Cómo están presentados los comandos. Todo lo que verás en bloques `console` es una **sesión de terminal real**: las líneas que empiezan con `$` son lo que se escribe; las demás son lo que el sistema respondió, tal cual. Los valores que cambian en cada ejecución (fechas, identificadores, claves, algunas huellas) serán distintos si repites el ejercicio; lo que debe coincidir es el **patrón** que se explica debajo de cada salida.

Esta ERSO se apoya en la anterior. La ERSO 005 publica documentos *dentro* del registro (VDR) que construyó la ERSO 004. Aquí se explica lo necesario para seguirla sola, pero el montaje en detalle del registro está en el informe de la 004.

Contenido

Parte	Secciones	De qué trata
Resumen ejecutivo	—	El problema, la solución y el resultado en una página
I · Marco conceptual	1 a 4	Todos los términos, explicados en lenguaje sencillo, y el método de trabajo
II · Qué pide la ERSO	5 a 10	Capacidades, condiciones, descripción, pasos y criterios, frase por frase
III · El proyecto	11 a 13	Qué se construyó para esta ERSO y cómo funciona por dentro
IV · Laboratorios	14 a 22	Las pruebas materiales de cada paso y cada criterio, con cada comando y su respuesta
V · Preguntas y respuestas	23 y 24	Todas las preguntas de comprensión con su respuesta
VI · Cómo responder	25 y 26	Redacción modelo de cada criterio y lista de comprobación
VII · Límites y operación	27 a 29	Lo que no se demuestra, hallazgos, solución de problemas y operación diaria
Anexos	A a D	Scripts, autocomprobación, ejercicios y referencias

Resumen ejecutivo

El problema. Un DID por sí solo es solo un nombre. Para que alguien pueda **comprobar una firma** de una entidad necesita encontrar **su clave pública**, y para eso el nombre debe llevar a un **documento público**, el *DID Document*. La ERSO 005 pide que ese documento se pueda **construir, publicar en una dirección web segura (HTTPS) calculada a partir del propio DID, y servir**, con la clave en un formato estándar (*Multikey*), **sin exponer jamás claves privadas ni datos de personas**, y limitado al laboratorio de pruebas.

La solución. Se aprovechó el registro (VDR) de la ERSO 004 y se completaron las piezas que pide esta ERSO: una regla que convierte un `did:web` en su dirección (y viceversa), un constructor de documentos con la clave pública en Multikey, un **validador con lista blanca** que rechaza cualquier campo no autorizado (claves privadas, datos civiles, propiedades desconocidas), una **lectura pública de solo lectura** con encabezados que permiten comprobar el contenido, y un **mecanismo de confirmación** que lee de vuelta lo publicado y compara su huella.

El resultado.

#	Criterio de aceptación	Resultado	Prueba material (sección)
1	El documento se sirve por HTTPS en la URL calculada	 Cumplido	Criterio 1 (§18)
2	Contiene la clave pública correcta (documento publicado y comparación de hash)	 Cumplido	Criterio 2 (§19)
3	No expone claves privadas ni datos civiles	 Cumplido	Criterio 3 (§20)
4	Un <code>id</code> desajustado se rechaza y el camino base opera con la extensión desactivada	 Cumplido	Criterio 4 (§21)

Además se demostraron, paso a paso, la **generación del identificador** (§16) y la **construcción del documento** (§17), que son los pasos 1 y 2 del PDF.

Lo que conviene saber con honestidad (detalle en la Parte VII). Una consulta `HEAD` al documento responde 404 aunque `GET` funciona (hallazgo menor que ningún criterio exige); el laboratorio usa una autoridad de certificación propia y no se probó en el dominio real; Base58 y Multikey son implementación propia sin contrastar con vectores de terceros; y el documento general del equipo usa otro formato de clave (`JsonWebKey2020`) que la ERSO no permite.

PARTE I — MARCO CONCEPTUAL: los términos que necesitas

💡 **Idea clave de esta parte.** Todo el proyecto descansa en pocas ideas. Si las entiendes aquí, el resto del informe es aplicarlas. Cada término está explicado con una frase sencilla, para qué sirve y dónde lo verás en este proyecto.

1. La historia en cinco minutos

Imagina que quieres comprobar que una carta realmente la firmó el rector de una universidad.

- Podrías **llamar al rector** cada vez. Funciona, pero depende de que esté disponible, y si lo hacen millones de personas, es inviable.
- O puedes tener a mano **una muestra confiable de su firma**, y comparar. Para que sirva, esa muestra debe estar en un lugar **público, estable y que nadie pueda alterar a escondidas**.

En el mundo digital pasa lo mismo:

- La "firma" del rector es una **firma digital**, hecha con una **clave privada** que solo él tiene.
- La "muestra confiable de su firma" es su **clave pública**, que cualquiera puede usar para **verificar** (no para falsificar) esa firma.
- El "lugar público, estable y a prueba de alteraciones" es el **registro verificable de datos** (VDR). **Eso es lo que esta ERSO construye.**
- El rector se identifica con un nombre llamado **DID**, y su "muestra de firma" vive en un archivo llamado **DID Document**.

💡 **En una frase:** el VDR es la agenda pública y confiable donde cualquiera busca la clave pública de una entidad.

2. Diccionario de términos

Los términos están agrupados por tema, en el orden en que conviene aprenderlos.

2.1 Criptografía básica

Término	Qué es (en palabras simples)	Para qué sirve	Dónde lo verás aquí
Hash (o huella)	Un "resumen" de longitud fija que se calcula a partir de cualquier contenido. El mismo contenido da siempre el mismo resumen; si cambias una sola letra , el resumen cambia por completo. No se puede deshacer para recuperar el contenido	Comprobar que algo no fue alterado	sha256:cb5799ba... en cada versión de un documento

Término	Qué es (en palabras simples)	Para qué sirve	Dónde lo verás aquí
SHA-256	El método concreto para calcular el hash que usa el proyecto. Produce una huella de 64 caracteres	Lo mismo	El prefijo <code>sha256:</code>
Clave privada	Un número secreto que solo su dueño conoce	Firmar	Nunca aparece en ninguna salida pública
Clave pública	Un número matemáticamente ligado a la privada, que se reparte libremente	Verificar firmas; no sirve para firmar	El <code>publicKeyMultibase</code> del DID Document
Par de claves	La privada y su pública, juntas	Identificarse sin compartir el secreto	Cada entidad tiene su par
Firma digital	El resultado de "firmar" un contenido con la clave privada	Demostrar quién lo firmó y que no cambió	La "prueba de posesión"
Verificar	Comprobar una firma usando la clave pública	Que cualquiera valide sin poder falsificar	El consumidor de DID (otra ERSO)
P-256 / ES256	El tipo de clave (una "curva elíptica") y el algoritmo de firma. Son compactos y seguros	Firmas de 64 bytes	Todas las claves del proyecto
Nonce	Un valor aleatorio de un solo uso	Impedir que una firma capturada se reutilice	El "desafío"
Codificación (Base64url, Base58)	Formas de escribir datos binarios como texto	Meterlos en JSON y URL	Las firmas y las claves
Multikey / multibase	Una convención para escribir una clave pública como texto, indicando su tipo. Las claves P-256 empiezan por <code>zDn</code>	Que cualquiera sepa leer la clave	<code>"publicKeyMultibase": "zDnae..."</code>
JSON canónico	Una forma de escribir un JSON siempre igual (claves ordenadas, sin espacios)	Que el hash y la firma sean reproducibles	El documento publicado

⚠ **Cuatro verbos que se confunden:** *codificar* (cambiar la forma, no es secreto), *hashear* (sacar la huella, no se revierte), *cifrar* (ocultar con una clave) y *firmar* (probar autoría). **Firmar no oculta nada:** el contenido sigue a la vista.

2.2 Redes y canales seguros

Término	Qué es	Para qué sirve	Dónde lo verás
Dominio / DNS	El nombre (<code>civica-desarrollo.avance.org.co</code>) y el sistema que lo traduce a una dirección	Que usemos nombres y no números	El DID es el dominio
HTTP / HTTPS	El protocolo de la web; HTTPS es HTTP protegido con TLS	Pedir y recibir documentos	GET <code>.../did.json</code>

Término	Qué es	Para qué sirve	Dónde lo verás
Puerto	Un número que distingue las "puertas" de una máquina	Separar servicios	8443, 9443, 8444
TLS	La capa que da a HTTPS tres cosas: cifrado (nadie escucha), integridad (nadie altera) y autenticidad del servidor	Comunicación segura	Todo el tráfico
Certificado	Un documento que dice "esta clave pública pertenece a este dominio", firmado por una autoridad	Que el cliente sepa con quién habla	<code>server.crt</code>
CA (autoridad de certificación)	Quien firma certificados; el cliente confía en unas pocas CA	Delegar la confianza	<code>ca.crt</code> (propia de laboratorio)
Cadena de confianza	Certificado → firmado por la CA → en la que el cliente confía	Validar un certificado	<code>openssl verify</code>
mTLS	TLS mutuo : además del servidor, el cliente también presenta certificado	Saber qué entidad está escribiendo	El canal de escritura
Proxy inverso (nginx)	Un servidor que recibe el tráfico y lo reenvía a los servicios internos	Poner TLS, separar canales y esconder el resto	<code>deploy/nginx/nginx.conf</code>
Docker	Una herramienta para ejecutar programas en contenedores aislados	Aislar y repetir el despliegue	Todo el proyecto
Imagen / contenedor	La "receta empaquetada" de un programa / su ejecución	Ejecutar igual en cualquier máquina	<code>vdr-ssi/vdr-service:local</code>
Red de Docker	Una red privada entre contenedores	Que se hablen sin salir al exterior	<code>vdr-ssi_default</code>
Docker Compose	Un archivo que describe todos los contenedores y cómo se conectan	Levantar el proyecto con un comando	<code>deploy/docker-compose.yml</code>
Puerto publicado	Un puerto del contenedor abierto hacia tu máquina	Ser alcanzable desde fuera	Solo nginx publica

2.3 Quién puede escribir (autenticación y permisos)

Término	Qué es	Dónde lo verás
Autenticación	Comprobar quién eres	Certificado y secreto
Autorización	Decidir qué puedes hacer	Cada entidad solo escribe en su espacio
OAuth2 <code>client_credentials</code>	Una máquina cambia su identificador y secreto por un token	<code>POST /admin/v1/oauth/token</code>
Token / JWT	Un permiso temporal (10 minutos) firmado por el servidor	<code>access_token</code>
JWS	El formato <code>cabecera.contenido.firma</code> para firmar datos	Todas las "pruebas"

Término	Qué es	Dónde lo verás
Desafío (<i>challenge</i>)	El servidor entrega un nonce y pide que se firme	POST /challenges
Prueba de posesión	Demostrar que posees la clave privada (no solo que sabes algo) firmando el desafío	Una escritura válida
Replay (repetición)	Reenviar algo que se capturó antes	El ataque que frena el nonce de un solo uso
Idempotencia	Repetir una petición no duplica el efecto	Cabecera Idempotency-Key
Concurrencia optimista	"Escribo solo si la versión actual es la N"	Cabecera If-Match
Namespace	El espacio del DID reservado a una entidad (entidades/avance)	Permisos por entidad

Códigos HTTP que aparecen (la respuesta numérica del servidor):

Código	Significa	Ejemplo aquí
200 / 201	Bien / creado	Escritura confirmada
400	Solicitud mal formada	nginx exige certificado cliente y no lo recibe
401	No sé quién eres	Falta el token
403	Sé quién eres, pero no tienes permiso	Firma inválida
404	No existe	DID inexistente, o ruta que no existe
409	Conflicto con el estado actual	Crear un DID que ya existe, o restaurar sobre datos vivos
410	Existió, pero ya no (a propósito)	DID desactivado
412	Falla una precondition	If-Match con la versión equivocada
422	Entendible pero inválido	Respaldo alterado, documento que no cumple el perfil
502	El servidor de entrada no encuentra al de atrás	nginx cuando el VDR está detenido

2.4 Identidad descentralizada

Término	Qué es	Dónde lo verás
Identificador	Un nombre único para algo	—
DID	Un identificador con la forma did:<método>:<resto> que su dueño controla sin una autoridad central	did:web:civica-desarrollo.avance.org.co:entidades:avance
Método DID	La "receta" que dice dónde y cómo viven los DID de ese tipo (en una web, en una cadena de bloques...)	La palabra web

Término	Qué es	Dónde lo verás
<code>did:web</code>	El método cuyo registro es un servidor web : el DID se convierte en una dirección	Todo el proyecto
DID Document	Un archivo JSON público con las claves públicas del DID y para qué sirve cada una	<code>.../did.json</code>
<code>verificationMethod</code>	La lista de claves públicas dentro del documento	<code>"type": "Multikey"</code>
<code>authentication</code>	Qué claves sirven para identificarse	Dentro del documento
<code>assertionMethod</code>	Qué claves sirven para firmar afirmaciones (por ejemplo, credenciales)	Dentro del documento
DID URL	Un DID más <code>#algo</code> , que apunta a una parte	<code>...#key-1</code>
Resolver	Dado un DID, obtener su documento	Otra ERSO (consumidor)
VDR	<i>Verifiable Data Registry</i> : el lugar donde se publican y consultan los DID Documents. Es un rol, no una tecnología	<code>vdr-service</code>

💡 **Tres errores comunes:** un DID **no es** necesariamente una cadena de bloques; un DID Document **no es** una credencial (dice qué claves tiene un identificador, no datos de una persona); y poseer una clave **no equivale** a ser una persona.

2.5 Datos, historial y recuperación

Término	Qué es	Dónde lo verás
PostgreSQL	La base de datos que guarda el estado	Contenedor <code>postgres</code>
Tabla / fila / clave primaria	La estructura de los datos / un registro / el campo que lo identifica	<code>did_documents</code> (clave: el DID)
Transacción	Un conjunto de cambios que se aplican todos o ninguno	Nunca queda el registro a medias
Estado actual e historial	La "foto de hoy" y el "álbum" de todas las versiones	<code>did_documents</code> y <code>did_document_versions</code>
Versión	Un número que crece con cada cambio	<code>v1</code> , <code>v2</code> ...
Append-only (solo se agrega)	Una tabla donde nada se edita ni se borra	El historial
Trigger	Una regla dentro de la base que se dispara sola	Lo que bloquea <code>DELETE</code> y <code>UPDATE</code>
Auditoría	El diario de quién hizo qué y cuándo, incluidos los intentos rechazados	<code>audit_log</code>

Término	Qué es	Dónde lo verás
Respaldo (backup)	Una copia completa del estado, guardada fuera	POST /admin/v1/backup
Checksum	El hash de un archivo completo	Detecta que un respaldo fue alterado
Restauración	Volver a cargar un respaldo	POST /admin/v1/restore
TRUNCATE	Vaciar una tabla por completo	Para simular la pérdida total

2.6 Palabras propias de las ERSO

Término	Qué es
ERSo	<i>Especificación de Requisitos de Software</i> : el documento que dice qué debe cumplir un desarrollo (capacidades, condiciones, pasos, criterios)
Capacidad	Algo que el sistema debe poder hacer
Condición	Una restricción que el diseño debe respetar siempre
Criterio de aceptación	Una condición medible que se debe cumplir, con la evidencia que la demuestra
Camino base	Lo que el piloto hace sin la extensión; debe seguir funcionando siempre
Marca "O"	<i>Opcional</i> : fuera del alcance base y desactivada por defecto
Regresión	Comprobar que lo que ya funcionaba sigue funcionando tras un cambio
Falla cerrada (<i>fail-closed</i>)	Ante la duda, lo riesgoso queda apagado o se rechaza
Falla temprana (<i>fail-fast</i>)	Es mejor no arrancar con una configuración peligrosa que arrancar y avisar
Prueba automática	Un programa que comprueba otro programa; repetible y rápido
Prueba de extremo a extremo (E2E)	Una prueba que recorre el sistema real (con red, TLS y certificados)
Evidencia	Lo que demuestra un criterio: una prueba, un registro, una salida de terminal

2.7 Términos propios de la publicación del documento

Término	Qué es (en palabras simples)	Para qué sirve	Dónde lo verás
Ruta calculada	La dirección web que se deduce del DID aplicando una regla fija	Que cualquiera encuentre el documento sin preguntar a nadie	https://.../lab/prueba/did.json
Segmento	Cada trozo de la ruta entre barras (lab , prueba-005)	Se valida que solo tenga letras, números, . , _ y -	El rechazo de . . y de espacios

Término	Qué es (en palabras simples)	Para qué sirve	Dónde lo verás
<code>.well-known</code>	Una carpeta reservada por convención para datos públicos de un dominio	Aloja el DID "raíz" de un dominio (<code>did:web:dominio</code>)	<code>/.well-known/did.json</code>
<code>%3A</code>	La forma de escribir el carácter <code>:</code> dentro de un DID cuando es parte del puerto	Distinguir el puerto del separador de rutas	<code>did:web:dominio%3A8443:...</code>
Normalización	Dejar un valor en su forma canónica (por ejemplo, dominio en minúsculas)	Que dos escrituras equivalentes den el mismo DID	<code>CIVICA-...</code> → <code>civica-...</code>
<code>@context</code>	Una lista de direcciones que dicen "este JSON usa el vocabulario DID"	Que el documento sea reconocible como DID Document	Primera propiedad del documento
Propiedad	Un campo del JSON (<code>id</code> , <code>verificationMethod</code> ...)	—	Las cinco que lleva el documento
Lista blanca	Solo se acepta lo que está en una lista autorizada; todo lo demás se rechaza	Es más seguro permitir lo conocido que intentar prohibir lo peligroso	<code>UNKNOWN_PROPERTY</code>
Datos civiles	Datos de una persona (nombre, fecha de nacimiento, correo...)	No deben ir nunca en un documento público	<code>CIVIL_DATA</code>
Correlación	Reconocer a la misma persona en contextos distintos por un identificador que se repite	Es el riesgo de publicar un DID permanente de cada ciudadano	Condición 3 de la ERSO
Content-Type: <code>application/did+json</code>	La etiqueta que dice "esto es un DID Document en JSON"	Que el cliente lo interprete bien	Encabezado de la respuesta
ETag	Un identificador de la versión servida	Saber si el documento cambió sin descargarlo	ETag: <code>"1"</code>
Cache-Control: <code>no-cache</code>	"Antes de reutilizar una copia, pregunta si sigue vigente"	Que una clave rotada o un DID desactivado no se sirvan desde una copia vieja	Encabezado de la respuesta
X-Content-Hash	Un encabezado propio con el hash del contenido servido	Comparar contenido y hash sin recalcular	Encabezado de la respuesta

Término	Qué es (en palabras simples)	Para qué sirve	Dónde lo verás
nosniff	Una orden al navegador: "no adivines el tipo de contenido"	Evitar que un archivo se interprete como otra cosa	X-Content-Type-Options: nosniff
GET / HEAD / POST / PUT	Los "verbos" de HTTP: leer / leer solo los encabezados / enviar / reemplazar	La puerta pública solo debe permitir leer	GET sí; POST y PUT no
Base58btc y multicodec	La forma de escribir bytes con 58 caracteres, y el prefijo que indica qué tipo de clave son	Escribir una clave P-256 como zDn...	El prefijo de bytes 0x8024
Punto comprimido	Una clave P-256 se guarda como un byte de paridad (02 o 03) más la coordenada x (32 bytes)	Que la clave ocupe 33 bytes y no 65	Laboratorio B
ION y anclaje	ION es un método DID que ancla sus registros en una cadena de bloques (Bitcoin)	No se integra en esta ERSO	Condición 1
Operador del dominio	Quien controla el DNS y el servidor web del dominio	En did:web la confianza depende de él	Límites (\$27)

3. Las siete ideas madre

#	Idea	En una frase
1	Confianza por verificación, no por autoridad	En lugar de "confío porque lo dice X", "confío porque puedo comprobarlo"
2	La pública verifica; la privada firma	La privada jamás sale de su dueño
3	Un hash es una huella	Cambia un byte y la huella cambia por completo
4	Leer es libre; cambiar exige demostrar	Cualquiera lee un documento; escribirlo exige canal seguro, permiso y prueba de la clave
5	Defensa en capas	Si una capa falla, quedan las otras
6	Cada rol tiene su clave y su canal	Nadie comparte puertas
7	Lo opcional está apagado y aislado	La extensión no puede romper lo que ya funcionaba

4. El método de trabajo: cómo se piensa un criterio

Para pasar de una frase del PDF a una prueba que otra persona pueda repetir, se siguió siempre el mismo ciclo, en dos niveles.

Para aprender cada tema (cinco pasos): *concepto* → *observación* → *predicción* → *correlación* → *evidencia*. Primero entiendes la idea, luego la ves funcionar, **predices** qué pasará antes de mirar (aquí es donde se aprende de verdad), conectas lo visto con el diseño y, al final, sabes demostrarlo.

Para resolver un criterio (siete pasos):

#	Paso	La pregunta
1	Descomponer	¿Cuál es el verbo, el objeto y la evidencia que pide?
2	Hacerlo comprobable	¿Qué debería ser cierto y cómo lo vería un tercero?
3	Buscar el caso negativo	¿Cómo intentaría romperlo un atacante o un error?
4	Elegir el mecanismo	¿Qué pieza lo garantiza? ¿Puede ser una capa baja (base de datos, proxy)?
5	Implementar y probar a la vez	¿Cómo falla esta prueba si el mecanismo no existe?
6	Doble evidencia	Prueba automática y prueba por la red real
7	Declarar los límites	¿Qué no se probó o está simulado?

💡 **El verbo manda.** "Se rechaza" pide una prueba negativa. "Queda registrada" pide mostrar el dato guardado. "Opera sin..." pide una regresión. Y **sin caso negativo, lo que tienes es una demostración, no una prueba.**

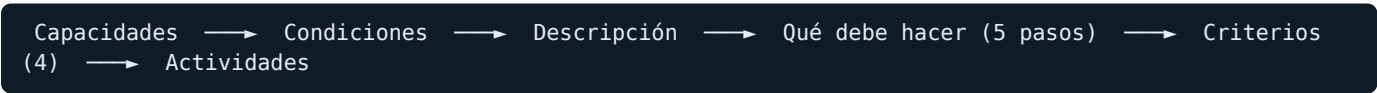
PARTE II — QUÉ PIDE LA ERSO 005, EXPLICADO FRASE POR FRASE

5. La ficha del documento

Campo del PDF	Valor
Proyecto	Identidad Digital Soberana SSI
Título	Publicación del DID Document bajo did:web
Desarrollador	Geovani Rincón
Fecha de la ERSO	Viernes, 18 de septiembre de 2026
Fecha de desarrollo	Miércoles, 30 de septiembre de 2026, 7:30 a. m.
Fecha de prueba	Viernes, 2 de octubre de 2026, 7:30 a. m.
Responsables de las actividades	Análisis y diseño, asignar desarrollador, aceptar archivos y archivo: Rocío Villamizar · Implementar: Geovani Rincón · Plantilla de pruebas y pruebas funcionales: Luis González

⚠ Las firmas y fechas de la tabla de actividades del PDF **corresponden a las personas responsables**; este informe aporta las **evidencias técnicas**, no las firmas.

El PDF se organiza en los mismos bloques que cualquier ERSO, y este informe los recorre en el mismo orden:



6. Capacidades del proceso

6.1 La frase principal, desarmada

"Se requiere habilitar **de forma opcional** la **publicación** y el **servicio web** de un **DID Document** mediante el método **did:web**, con **clave P-256 convertida a Multikey**, **sin alterar el camino base** del piloto."

#	Fragmento	Qué significa en lenguaje sencillo	Cómo se materializó
1	"de forma opcional"	La publicación solo existe si se enciende la extensión; por defecto está apagada	Es parte del VDR de la ERSO 004 (VDR_ENABLED)
2	"la publicación y el servicio web"	Dos mitades: subir el documento y entregarlo a quien lo pida por la web	Escritura autenticada (PUT) y lectura pública (GET */did.json)

#	Fragmento	Qué significa en lenguaje sencillo	Cómo se materializó
3	"de un DID Document"	El archivo JSON con las claves públicas del identificador	Construido con <code>DidDocumentBuilder</code>
4	"mediante el método <code>did:web</code> "	El DID se convierte en una dirección web: el "registro" es un servidor web	<code>DidWeb.kt</code>
5	"con clave P-256 convertida a Multikey"	La clave pública es de tipo P-256 y se escribe en el formato estándar <code>zDn...</code>	<code>Multikey.kt</code>
6	"sin alterar el camino base"	Lo que ya funcionaba sigue funcionando	Criterio 4 (regresión)

💡 **Qué se busca, en una frase:** que un identificador `did:web` tenga **un documento público, bien formado y sin secretos, disponible en una dirección HTTPS que cualquiera pueda calcular**, y que eso no perturbe nada más.

6.2 Las cuatro capacidades

"Entre las capacidades del proceso se encuentran: generar un identificador `did:web` conforme al método y al dominio configurado; construir el DID Document con la clave pública P-256 en representación Multikey; publicar el documento en la ruta web calculada y servirlo por HTTPS; comprobar la publicación mediante lectura desde la URL pública y comparación de contenido/hash."

#	Capacidad	Qué se busca	Cómo se resolvió	Paso del PDF
1	Generar el identificador <code>did:web</code>	Que el DID salga del dominio configurado, con reglas claras y seguras	<code>DidWeb.of(dominio, segmentos)</code> y <code>DidWeb.url(did)</code>	Paso 1
2	Construir el documento con Multikey	Un JSON con la clave pública y para qué sirve, y nada más	<code>DidDocumentBuilder.build</code> + <code>Multikey.encodeP256</code>	Paso 2
3	Publicar y servir por HTTPS	Que el documento quede en la ruta calculada y se entregue con TLS	Escritura en el registro; lectura por nginx con TLS	Paso 3
4	Comprobar leyendo la URL y comparando hash	No creerse a uno mismo: leer lo publicado como lo haría un tercero	Confirmación automática del registro + comprobación manual	Paso 4

7. Condiciones del proceso

Una **condición** es una **restricción** que el diseño debe respetar siempre. Para cada una conviene preguntarse **qué problema evita**.

Condición 1 — "El método admitido es `did:web`; no se integra ION ni anclajes equivalentes."

- **Qué significa.** Solo `did:web`. No se usan métodos que guarden sus registros en una **cadena de bloques** (ION los ancla en Bitcoin) ni mecanismos de "anclaje" equivalentes.
- **Qué problema evita.** Complejidad y dependencias innecesarias para el piloto, y confundir "registro verificable" con "cadena de bloques".
- **Cómo se cumple.** Cualquier DID de otro método se rechaza (`invalidDid`: El DID debe usar el método `did:web`).
- **⚠ Contrapartida honesta:** `did:web` hereda las debilidades de la web: depende del **DNS**, del **HTTPS** y del **operador del dominio**. El PDF lo advierte expresamente.

Condición 2 — "La aceptación se limita al laboratorio de pruebas de identidad."

- **Qué significa.** Esta ERSO se prueba **en el entorno de laboratorio**, no en producción ni con datos reales.
- **Qué problema evita.** Exponer algo no probado a ciudadanos o entidades reales.
- **Cómo se cumple.** Todo corre con una autoridad de certificación propia y con la entidad de laboratorio (`lab-operator`) y su espacio `lab/...`.

Condición 3 — "No se publica por defecto ningún DID permanente del ciudadano."

- **Qué significa.** Nadie debe quedar con un identificador público y **duradero** por el solo hecho de existir.
- **Qué problema evita.** La **correlación**: si cada persona tuviera un DID público fijo, cualquier verificador podría reconocerla en contextos distintos y seguirle el rastro.
- **Cómo se cumple.** Solo se pueden publicar DID dentro de **espacios reservados a una entidad**. La entidad de laboratorio **no puede** crear `...:ciudadanos:ana` (se verá en §21).
- **⚠ Nota de coherencia con otra ERSO.** Más adelante, la ERSO 003 permite que el *Wallet Backend* publique el DID del titular bajo `titulares/<huella>`, por decisión del diagrama de arquitectura. Eso es un camino **distinto y explícito**; la condición 3 de esta ERSO se sigue cumpliendo para el flujo que ella describe (el laboratorio). El riesgo de correlación de ese otro camino está documentado en el informe de la ERSO 003.

8. Descripción del proceso

Es el **porqué** de la ERSO. Tiene cuatro ideas importantes.

"Publicar un DID Document significa dejarlo disponible en una URL HTTPS para que cualquiera pueda resolverlo y obtener las claves públicas del sujeto."

- "Cualquiera": la lectura es **pública y sin autenticación**. "Resolverlo": pedir el DID y obtener el documento.

"Esta ERSO usa el método `did:web`, donde la URL se deriva del dominio, y clave P-256 expresada como Multikey."

- "Se deriva del dominio": no hay que registrarse en ninguna parte; el DID **ya contiene** la dirección.

"No conviene publicar por defecto un DID de cada ciudadano: un identificador público persistente facilita la correlación entre contextos de uso, y el piloto no necesita ese punto de exposición."

- Es la razón de la condición 3.

"También debe tener presente que `did:web` depende del DNS, del HTTPS y del operador del dominio."

- Quien controle el dominio **controla** lo que se publica. Eso es un riesgo aceptado, no resuelto.

"Literatura y temas a consultar" — la lista de estudio que asigna la ERSO:

Lectura	Qué te aporta	Para qué la usarías
W3C DIDs v1.1	El modelo de datos del documento: <code>id</code> , <code>verificationMethod</code> , relaciones	Saber qué debe y qué no debe llevar el documento
Especificación del método <code>did:web</code>	La regla exacta DID → URL, incluido el DID raíz (<code>.well-known</code>) y el puerto	Paso 1
Multikey / multibase	Cómo se escribe una clave pública P-256 como texto	Paso 2
Ejemplos de publicación de <code>did.json</code> y buenas prácticas de servido estático	Tipo de contenido, caché, rutas	Paso 3

Los enlaces están en `docs/VERSIONES-NORMATIVAS.md`.

9. Qué debe hacer: los cinco pasos

1. Generar el identificador `did:web` conforme al método y al dominio configurado.
2. Construir el DID Document con la clave pública P-256 en Multikey, **limitado al material público de verificación, relaciones de uso de claves y servicios opcionales permitidos**.
3. Publicar el documento en la ruta web calculada y servirlo por HTTPS.
4. Comprobar la publicación leyendo desde la URL pública y comparando contenido y hash.
5. Asegurar que el documento no incluye credenciales personales, claves privadas ni datos civiles.

Paso	Lo que pide	Capacidad / condición	Criterio que lo comprueba	Laboratorio
1	Generar el <code>did:web</code>	Capacidad 1 · Condición 1	Indirecto (la URL calculada del criterio 1)	\$16

Paso	Lo que pide	Capacidad / condición	Criterio que lo comprueba	Laboratorio
2	Construir el documento (solo público)	Capacidad 2	Criterio 2 (clave correcta) y 3	\$17
3	Publicar y servir por HTTPS	Capacidad 3	Criterio 1	\$18
4	Comprobar leyendo y comparando hash	Capacidad 4	Criterio 1 y 2	\$18 y \$19
5	Sin credenciales, claves privadas ni datos civiles	Condición 3	Criterio 3	\$20

💡 **Dos observaciones.** (1) El **criterio 4** no sale de un paso concreto: añade lo que la ERSO exige *además* —el rechazo de un `id` desajustado y la regresión con la extensión apagada—. (2) El **paso 1** no tiene criterio propio, así que se aporta evidencia adicional (§16).

10. Los cuatro criterios de aceptación

Criterio 1. "El documento se sirve por HTTPS en la URL calculada; evidencia: lectura desde la URL pública."

Criterio 2. "El documento contiene la clave pública correcta; evidencia: documento publicado y comparación de hash."

Criterio 3. "El documento no expone claves privadas ni datos civiles; evidencia: revisión del contenido publicado."

Criterio 4. "Un `id` desajustado se rechaza y el camino base opera con la extensión desactivada; evidencia: pruebas negativas y de regresión."

Aplicando los primeros pasos de la receta (descomponer, hacerlo comprobable, buscar el caso negativo):

Criterio	El verbo y la evidencia	Lo que debo poder mostrar	El caso negativo	Mecanismo que lo garantiza
1	Se sirve · lectura desde la URL pública	Un <code>GET</code> por HTTPS a la ruta calculada devuelve el documento con el tipo correcto	Otra ruta, otro verbo, o un DID inexistente no lo devuelven	Regla DID→URL; nginx solo deja pasar <code>GET / HEAD</code> de <code>*/did.json</code>
2	Contiene la clave correcta · documento publicado y hash	La clave publicada es la misma que se generó, y el hash del contenido servido es el del registro	Cambiar un carácter cambia el hash; una clave ajena no verifica mi firma	Mismo JSON canónico en escritura y lectura; verificación con la clave publicada
3	No expone · revisión del contenido	El documento tiene solo las propiedades autorizadas y ningún secreto	Intentar publicar <code>privateKeyMultibase</code> , <code>credentialSubject</code> , <code>birthdate</code> , una propiedad propia o un servicio <code>http</code>	Validador con lista blanca

Criterio	El verbo y la evidencia	Lo que debo poder mostrar	El caso negativo	Mecanismo que lo garantiza
4	Se rechaza y opera · pruebas negativas y de regresión	Un documento cuyo id no es el DID no se publica; con la extensión apagada el camino base responde	Documentos con id ajeno; DID de ciudadano; dominio ajeno; segmento ..	ID_MISMATCH ; precondiciones antes de entregar un desafío; rutas inexistentes si está apagado

PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO FUNCIONA

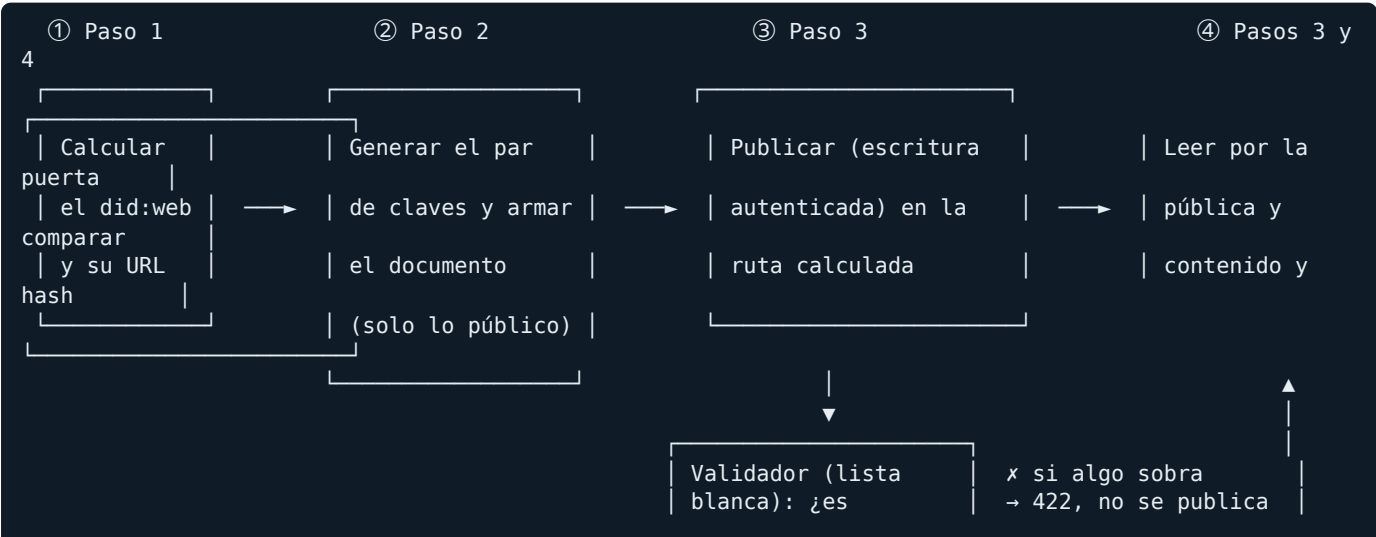
11. Visión general

La ERSO 005 **no levanta un sistema nuevo**: completa y usa el registro (VDR) de la ERSO 004. Lo que aporta es todo lo necesario para **producir, publicar y servir un DID Document**.

11.1 Las piezas que intervienen

Pieza	Qué es	Qué hace en la ERSO 005
did-core	Una librería de reglas del DID	Convierte un did:web en su dirección y al revés (DidWeb), codifica claves en Multikey (Multikey, Base58), arma el documento (DidDocumentBuilder), lo escribe siempre igual (CanonicalJson) y lo valida con lista blanca (DidDocumentValidator)
vdr-service	El servicio del registro	Recibe la publicación autorizada, valida el documento, lo guarda y lo sirve por la puerta pública con los encabezados correctos
PostgreSQL	La base de datos	Guarda el documento y su historial
nginx	El servidor de entrada	Pone TLS, deja pasar solo GET/HEAD de */did.json por la puerta pública, y agrega nosniff
did-tools	Un programa de línea de comandos	Actúa como "la entidad": genera claves, calcula DID, arma documentos, firma y publica. Es lo que usamos en los laboratorios
Docker Compose	El orquestador	Mantiene todo levantado (ver el informe de la ERSO 004, §12)

11.2 El recorrido de un documento, de punta a punta



vuelta)

aceptable?

(la confirmación lee de

💡 **Lo esencial:** el documento pasa por **tres filtros** antes de existir públicamente —la regla del identificador, el validador de contenido y la prueba de posesión de la clave— y **después** se comprueba leyéndolo como lo haría un tercero.

12. El entorno para esta ERSO

🔍 **Qué vas a hacer.** Verificar que el registro está levantado, ver las dos piezas de configuración que importan para esta ERSO (la puerta pública de nginx y la ruta que sirve el documento), y comprobar el espacio reservado para el laboratorio.

⚠️ **Si el proyecto está detenido,** levántalo antes de seguir: `cd deploy && docker compose up -d`. El montaje completo (certificados, compilación, imágenes) está explicado paso a paso con sesión de terminal en el informe de la ERSO 004, §12. Aquí solo se comprueba lo necesario.

12.1 Entrar a la carpeta de despliegue y preparar el terminal

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect/deploy
$ set -a; . ./env; set +a; export LOCAL_UID=$(id -u) LOCAL_GID=$(id -g)
$ D=$VDR_DOMAIN
$ PUB="--cacert certs/ca.crt --resolve $D:8443:127.0.0.1"
$ W=../evidencias/work
$ PSQL="docker compose exec -T postgres psql -U $POSTGRES_USER -d $POSTGRES_DB"
$ TOOLS() { docker compose --progress quiet --profile tools run --rm -T tools "$@"; }
$ LABOP() { TOOLS admin --admin-url "https://$D:8443" --client-id lab-operator --secret
"$CLIENT_SECRET_LAB" --p12 /certs/lab-operator.p12 --p12-pass "$P12_PASS" --ca /certs/ca.crt "$@"; }
$ PUBLICAR() { local did=$1; shift; TOOLS write --admin-url "https://$D:8443" --client-id lab-
operator --secret "$CLIENT_SECRET_LAB" --p12 /certs/lab-operator.p12 --p12-pass "$P12_PASS" --ca
/certs/ca.crt --did "$did" "$@"; }
```

Comando	Qué hace
<code>cd ../deploy</code>	Entra a la carpeta donde están el <code>docker-compose.yml</code> y los certificados
<code>set -a; . ./env;</code> <code>set +a</code>	Carga en el terminal las variables de <code>deploy/.env</code> (dominio, contraseñas...)
<code>export LOCAL_UID=...</code> <code>LOCAL_GID=...</code>	Para que los archivos que cree la herramienta dentro del contenedor te pertenezcan
<code>D=...</code>	El dominio del laboratorio
<code>PUB=...</code>	Las opciones de <code>curl</code> para la puerta pública : <code>--cacert</code> ("confía en nuestra CA de laboratorio") y <code>--resolve</code> ("ese dominio, en el puerto 8443, está en mi máquina")
<code>W=...</code>	La carpeta donde la herramienta deja los archivos de claves y documentos
<code>PSQL=...</code>	Atajo para ejecutar el cliente de PostgreSQL dentro del contenedor de la base
<code>TOOLS()</code>	Ejecuta <code>did-tools</code> dentro de un contenedor (lo usamos para claves, documentos y firmas)

Comando	Qué hace
LABOP()	Una consulta de administración como <code>lab-operator</code> , la entidad del laboratorio
PUBLICAR()	Una escritura en el registro como <code>lab-operator</code> , sobre el DID que le indiques

12.2 Comprobar que el registro está vivo

```
$ docker compose ps --format 'table {{.Service}}\t{{.Status}}'
SERVICE      STATUS
credential     Up 3 minutes
nginx          Up 3 minutes
postgres       Up 3 minutes (healthy)
vdr            Up 3 minutes
wallet         Up 3 minutes
$ curl -s $PUB https://$D:8443/health
{"status":"UP","vdr":"enabled"}
```

✅ **Qué significa.** Cinco servicios Up (postgres con (healthy)) y {"status":"UP","vdr":"enabled"}: el registro está **encendido**.

12.3 La puerta pública de nginx

```
$ sed -n '/listen 443/,/location \/ { return 404; }/p' nginx/nginx.conf
listen 443 ssl;
server_name civica-desarrollo.avance.org.co localhost;
ssl_certificate /etc/nginx/certs/server.crt;
ssl_certificate_key /etc/nginx/certs/server.key;

add_header X-Content-Type-Options nosniff always;

location = /health { proxy_pass http://vdr:8080; }
location = /base/ping { proxy_pass http://vdr:8080; }

location ~ /did\.json$ {
    limit_except GET HEAD { deny all; }
    proxy_pass http://vdr:8080;
}

location / { return 404; }
```

✅ **Qué significa.** Es el bloque que atiende el puerto 443 del servidor (8443 en tu máquina). Lo importante:

Línea	Qué hace
<code>add_header X-Content-Type-Options nosniff always;</code>	Agrega <code>nosniff</code> a todas las respuestas
<code>location ~ /did\.json\$ { ... }</code>	Solo las rutas que terminan en <code>/did.json</code> se reenvían a la aplicación
<code>limit_except GET HEAD { deny all; }</code>	Cualquier verbo distinto de <code>GET</code> y <code>HEAD</code> se prohíbe (403)
<code>location / { return 404; }</code>	Todo lo demás responde 404

💡 Esta es la implementación de "la lectura pública es de solo lectura": no hay forma de escribir por esta puerta.

12.4 La ruta de la aplicación que sirve el documento

```
$ sed -n '/get("{path...}")/,/^    }$/p' ../vdr-
service/src/main/kotlin/co/org/avance/ssi/vdr/Server.kt
    get("{path...}") {
        val segments = call.parameters.getAll("path").orEmpty()
        val did = registry.didFromUrlPath(segments) ?: throw
ApiException(HttpStatusCode.NotFound, "NOT_FOUND", "Recurso no encontrado")
        when (val doc = registry.publicDocument(did)) {
            is PublicDoc.Found -> {
                call.response.header(HttpHeaders.ETag, "\"${doc.version}\"")
                call.response.header(HttpHeaders.CacheControl, "no-cache")
                call.response.header("X-Content-Hash", doc.hash)
                call.respondBytes(doc.bytes, DID_JSON, HttpStatus.OK)
            }
            PublicDoc.Gone -> throw ApiException(HttpStatus.Gone, "DEACTIVATED", "El DID
fue desactivado")
            PublicDoc.NotFound -> throw ApiException(HttpStatus.NotFound, "NOT_FOUND",
"El DID no existe")
        }
    }
}
```

✓ **Qué significa.** Cuando llega una petición:

Línea	Qué hace
registry.didFromUrlPath(segments)	Deshace la regla del método: de la ruta vuelve al DID. Si no termina en <code>did.json</code> , responde 404
registry.publicDocument(did)	Busca el documento vigente en la base
ETag, Cache-Control, X-Content-Hash	Agrega encabezados para comprobar el contenido y evitar copias viejas
respondBytes(doc.bytes, DID_JSON, 200)	Entrega exactamente los bytes guardados con el tipo <code>application/did+json</code>
PublicDoc.Gone → 410	Un DID desactivado ya no se sirve
PublicDoc.NotFound → 404	Un DID inexistente tampoco

⚠ **Fíjate** en que solo existe `get(...)`. No hay una ruta `head(...)`: es el origen del hallazgo de la sección 27.1.

12.5 El espacio reservado para el laboratorio

💡 **Para qué.** Publicar un DID exige que la entidad tenga **reservado** el espacio donde va a escribir (así una entidad no puede ocupar el de otra). Para que cada ejecución de los laboratorios pueda **publicar un DID nuevo** (en vez de actualizar siempre el mismo), se reservó para la entidad de laboratorio el espacio `lab/*`, además de `lab/laboratorio`. El `*` significa "cualquier nombre de un solo nivel" (por ejemplo, `lab/prueba-005-123`).

```
$ echo "select path, owner_client_id from namespaces where path like 'lab/%' order by path" | $PSQL
path                               | owner_client_id
-----+-----
lab/*                              | lab-operator
lab/laboratorio                    | lab-operator
lab/prueba-005-1790872156         | lab-operator
(3 rows)
```

✓ **Qué significa.** Dos reservas de `lab-operator`: la exacta `lab/laboratorio` y la general `lab/*`. Este cambio de configuración se hizo **solo para poder repetir los laboratorios**; está en `deploy/.env` y en `scripts/gen-env.sh`.

⚠ **Consecuencia práctica.** Cada vez que ejecutes los laboratorios se publicará un DID de prueba nuevo (`lab/prueba-005-...`) y **quedará en el registro** (un DID publicado solo puede desactivarse, no borrarse).

13. Cómo funciona por dentro (lo mínimo para entender las pruebas)

13.1 La regla que convierte un `did:web` en una dirección

Es el núcleo del paso 1 y se implementa en `DidWeb.kt`.

Regla	Ejemplo	Por qué
<code>did:web:</code> + dominio + <code>:</code> + segmentos	<code>did:web:civica-desarrollo.avance.org.co:lab:prueba</code>	Forma general
Los <code>:</code> después del dominio pasan a <code>/</code>	<code>.../lab/prueba</code>	Así se obtiene la ruta
Se añade <code>/did.json</code> al final	<code>.../lab/prueba/did.json</code>	Es el nombre del archivo
Se usa <code>https://</code>	<code>https://civica-desarrollo.avance.org.co/lab/prueba/did.json</code>	La lectura es siempre segura
Sin segmentos → <code>/.well-known/did.json</code>	<code>did:web:dominio</code>	El DID "raíz" del dominio
El puerto se escribe <code>%3A</code> en el DID	<code>did:web:dominio%3A8443:...</code> → <code>https://dominio:8443/...</code>	Para no confundir el puerto con un separador
El dominio se pasa a minúsculas	<code>CIVICA-...</code> → <code>civica-...</code>	Normalización
Cada segmento solo admite <code>A-Z a-z 0-9 . _ -</code> , y nunca <code>.</code> ni <code>..</code>	<code>..</code> y <code>mi ruta</code> se rechazan	Evita rutas que "suban" de carpeta y caracteres ambiguos

13.2 Qué se permite en un documento (la lista blanca)

El validador (`DidDocumentValidator`, perfil de publicación) **solo acepta** lo autorizado. Todo lo demás se rechaza con un código que dice por qué.

Control	Qué exige o prohíbe	Código si falla
Propiedades	Solo <code>@context</code> , <code>id</code> , <code>controller</code> , <code>verificationMethod</code> , <code>service</code> y las relaciones (<code>authentication</code> , <code>assertionMethod</code> , <code>keyAgreement</code> , <code>capabilityInvocation</code> , <code>capabilityDelegation</code>)	<code>UNKNOWN_PROPERTY</code>

Control	Qué exige o prohíbe	Código si falla
Claves privadas	Ningún campo con aspecto de clave privada (<code>d</code> , <code>privateKeyJwk</code> , <code>privateKeyMultibase</code> , <code>secretKey</code> ...), en cualquier profundidad	<code>PRIVATE_KEY_MATERIAL</code>
Datos civiles	Ningún campo como <code>credentialSubject</code> , <code>birthDate</code> , <code>name</code> , <code>email</code> , <code>address</code> , <code>nationalId</code> ...	<code>CIVIL_DATA</code>
id	Debe ser un DID <code>did:web</code> válido y igual al DID esperado	<code>INVALID_ID</code> , <code>ID_MISMATCH</code>
@context	El primer valor debe ser el contexto DID de W3C	<code>INVALID_CONTEXT</code>
Claves	Tipo <code>Multikey</code> , con una clave P-256 decodificable	<code>UNSUPPORTED_KEY_TYPE</code> , <code>INVALID_KEY</code>
Controladores	Solo el propio DID (sin controladores ajenos)	<code>UNAUTHORIZED_CONTROLLER</code>
Relaciones	Las claves de <code>authentication</code> y <code>assertionMethod</code> deben existir en <code>verificationMethod</code> ; debe haber al menos una en <code>authentication</code>	<code>DANGLING_REFERENCE</code> , <code>MISSING_AUTHENTICATION</code>
Servicios	Si hay, deben tener <code>id</code> , <code>type</code> y un <code>serviceEndpoint</code> que empiece por <code>https://</code>	<code>INVALID_SERVICE</code>
Tamaño	Máximo 64 KiB	<code>TOO_LARGE</code>

13.3 Por qué el documento se escribe "siempre igual" (JSON canónico)

Un mismo JSON puede escribirse de muchas formas (orden de campos, espacios) y cada forma tiene un **hash distinto**. El proyecto escribe el documento con **las claves ordenadas y sin espacios**, y guarda y sirve **exactamente esos bytes**. Resultado: *lo que se hashea, lo que se firma y lo que se sirve son lo mismo*, y por eso "comparar hash" es una comprobación exacta (criterio 2).

13.4 Mapa de requisitos a implementación

Requisito de la ERSO	Cómo se resolvió	Dónde (archivo)
Generar el <code>did:web</code> conforme al método y dominio	<code>DidWeb.of</code> , <code>DidWeb.url</code> ; dominio en <code>VDR_DOMAIN</code>	<code>did-core/.../DidWeb.kt</code>
Documento con clave P-256 en Multikey, solo público	<code>DidDocumentBuilder.build</code> , <code>Multikey.encodeP256</code>	<code>DidDocumentBuilder.kt</code> , <code>Multikey.kt</code>
Publicar y servir por HTTPS	Escritura autenticada; lectura pública tras <code>nginx</code> con TLS; <code>application/did+json</code> con <code>ETag</code>	<code>Server.kt</code> , <code>nginx.conf</code>
Comprobar leyendo la URL y comparando hash	Confirmación automática del registro + comprobación manual	<code>RegistryService.confirm</code> , laboratorios
Sin credenciales, claves privadas ni datos civiles	Validador con lista blanca	<code>DidDocumentValidator.kt</code>
Un <code>id</code> desajustado se rechaza	<code>ID_MISMATCH</code> (422)	<code>DidDocumentValidator.kt</code>

Requisito de la ERSO	Cómo se resolvió	Dónde (archivo)
Extensión desactivada ⇒ camino base intacto	Las rutas del registro no se registran si está apagado	Main.kt , Server.kt

PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA PASO Y CADA CRITERIO

14. Cómo funcionan los laboratorios

💡 Cada laboratorio tiene siempre la misma forma: **(1)** el objetivo, **(2)** unas predicciones para que pienses *antes* de ver el resultado, **(3)** la sesión de terminal real con cada comando y la respuesta del sistema, **(4)** la explicación de qué hace cada comando y qué se busca, **(5)** la lectura del resultado con las respuestas a las predicciones y **(6)** la conclusión: la **evidencia** que aporta.

Mapa de laboratorios

Sección	Laboratorio	Qué demuestra	Paso / criterio
§15	Preparación común	Un identificador de prueba único para toda la sesión	—
§16	A	Generar el <code>did:web</code> y su dirección	Paso 1
§17	B	Construir el documento (solo material público)	Paso 2
§18	C	Publicar y servir por HTTPS en la URL calculada	Criterio 1 (pasos 3 y 4)
§19	D	Clave pública correcta y comparación de hash	Criterio 2 (paso 4)
§20	E	Sin claves privadas ni datos civiles	Criterio 3 (paso 5)
§21	F	<code>id</code> desajustado rechazado y camino base con la extensión apagada	Criterio 4
§22	G	Las pruebas automáticas y el recorrido de extremo a extremo	Todos

⚠️ **Los laboratorios forman una cadena:** del B al F usan el mismo identificador y los mismos archivos de claves. Ejecútalos **en orden y en el mismo terminal** (las variables viven mientras no lo cierres). La forma más cómoda es el script `scripts/lab-005-completo.sh`, que los ejecuta todos en una sola sesión y produce lo mismo que este informe.

⚠️ **Requisitos:** el proyecto levantado (§12) y el espacio `lab/*` reservado (§12.5).

15. Preparación común: un identificador de prueba único

Objetivo. Elegir el nombre del DID que se va a publicar. Para poder repetir los laboratorios sin chocar con ejecuciones anteriores, el nombre incluye la hora actual.

```
$ ID=prueba-005-$(date +%s); DID=did:web:$D:lab:$ID; echo "ID=$ID"; echo "DID=$DID"
ID=prueba-005-1790872319
DID=did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319
```

Comando	Qué hace
ID=prueba-005-\$(date +%s)	Crea un nombre que termina en la hora actual en segundos (date +%s), así es único en cada ejecución
DID=did:web:\$D:lab:\$ID	Compone el DID completo: método web , el dominio, y la ruta lab / nombre

✓ **Lectura.** Quedan dos variables: ID (el nombre corto) y DID (el identificador completo). Todos los laboratorios siguientes las usan.

16. Laboratorio A — Paso 1: generar el did:web

"Generar el identificador did:web conforme al método y al dominio configurado."

Objetivo. Ver cómo el DID **sale del dominio y la ruta**, y cómo el sistema se protege de entradas peligrosas.

Predice antes de ejecutar

- **P1.** ¿Cuál será la dirección web (URL) del DID did:web:civica-desarrollo.avance.org.co:lab:<nombre> ?
- **P2.** Si el dominio lleva un **puerto** (:8443), ¿cómo se escribe en el DID y cómo en la URL?
- **P3.** Si escribes el dominio con **mayúsculas**, ¿qué pasa?
- **P4.** ¿Qué hará el sistema con una ruta que contenga **..** o un **espacio**?

16.1 Calcular el DID y su dirección

```
$ T00LS did --domain $D --namespace lab/$ID
did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319
https://civica-desarrollo.avance.org.co/lab/prueba-005-1790872319/did.json
$ T00LS did --domain "$D%3A8443" --namespace lab/laboratorio
did:web:civica-desarrollo.avance.org.co%3A8443:lab:laboratorio
https://civica-desarrollo.avance.org.co:8443/lab/laboratorio/did.json
$ T00LS did --domain CIVICA-Desarrollo.AVANCE.org.co --namespace lab/laboratorio
did:web:civica-desarrollo.avance.org.co:lab:laboratorio
https://civica-desarrollo.avance.org.co/lab/laboratorio/did.json
```

Comando	Qué hace	Qué se busca
T00LS did --domain \$D --namespace lab/\$ID	Pide a la herramienta que calcule el DID de la ruta lab/<nombre> y la dirección donde vive su documento	Ver la regla did:web en acción
... --domain "\$D%3A8443" ...	Lo mismo con un dominio que lleva puerto	Ver cómo se codifica
... --domain CIVICA-Desarrollo.AVANCE.org.co ...	Lo mismo con mayúsculas	Ver la normalización

✓ Lectura. Respuestas a P1, P2 y P3.

- **P1:** la primera línea es el DID y la segunda su URL: los `:` después del dominio pasaron a `/` y se añadió `/did.json`.
- **P2:** el puerto se escribe `%3A8443` **dentro del DID** (porque un `:` normal separaría segmentos) y `:8443` **en la URL**.
- **P3:** el dominio se convierte a **minúsculas**: el DID resultante es el mismo que con minúsculas.

16.2 Entradas peligrosas

```
$ T00LS did --domain $D --namespace "lab/../etc" 2>&1 | head -2
did:web:civica-desarrollo.avance.org.co:lab:../etc
Exception in thread "main" co.org.avance.ssi.didcore.InvalidDidException: Segmento de ruta inválido
en did:web
$ T00LS did --domain $D --namespace "lab/mi ruta" 2>&1 | head -2
did:web:civica-desarrollo.avance.org.co:lab:mi ruta
Exception in thread "main" co.org.avance.ssi.didcore.InvalidDidException: Segmento de ruta inválido
en did:web
```

Comando	Qué hace	Qué se busca
<code>--namespace "lab/../etc"</code>	Una ruta con <code>..</code> (en una web, "sube" una carpeta)	Que se rechace
<code>--namespace "lab/mi ruta"</code>	Una ruta con un espacio	Que se rechace
<code>2>&1 head -2</code>	Muestra solo las dos primeras líneas del resultado, incluidos los mensajes de error	Evitar un listado largo de errores técnicos

✓ **Lectura. Respuesta a P4.** La herramienta primero arma el texto del DID (por eso imprime la primera línea) y, al **validarlo** para calcular la URL, lo rechaza con `InvalidDidException: Segmento de ruta inválido en did:web`. Cada segmento solo admite letras, números y `.`, `_`, `-`, y nunca `.` ni `..`. 💡 *Por qué importa:* `..` podría usarse para salir de la carpeta que se quiere publicar. El **servidor** repite esta validación (se verá en §21 con el código `DID_INVALID`).

🎯 Conclusión del laboratorio A

Lo que pide el PDF	Evidencia
Identificador <code>did:web</code> conforme al método	El DID se calcula con la regla del método, incluido puerto y minúsculas (§16.1)
...y al dominio configurado	El dominio sale de la configuración (<code>VDR_DOMAIN</code>)
Seguridad de la entrada	Segmentos con <code>..</code> o espacios se rechazan (§16.2)
Prueba automática equivalente	Erso005DidWebTest > paso 1 - el identificador <code>did web</code> sale del dominio configurado (§22)

17. Laboratorio B — Paso 2: construir el documento

"Construir el DID Document con la clave pública P-256 en representación Multikey, limitado al material público de verificación, relaciones de uso de claves y servicios opcionales permitidos."

Objetivo. Generar un par de claves, armar el documento con la parte **pública**, y comprobar que el secreto **no entra**.

Predice antes de ejecutar

- **P1.** El archivo de claves que crea la herramienta, ¿qué campos tendrá?
- **P2.** ¿Qué propiedades tendrá el documento?
- **P3.** ¿Aparecerá alguna palabra como `privateKey` en el documento?
- **P4.** El texto `zDn...` de la clave pública, ¿qué contiene en realidad por dentro?

17.1 Generar la clave y ver qué guarda

```
$ T00LS keygen --out c005-clave.json
Clave P-256 generada. Multikey público: zDnaemzHWpsCu6cc4V5Er6reddfTqoGRiypwqp2mJgX4WHDfh
$ jq -r 'keys[]' $W/c005-clave.json
privateKeyPkcs8
publicKeyMultibase
```

Comando	Qué hace	Qué se busca
<code>T00LS keygen --out c005-clave.json</code>	Genera un par de claves P-256 y lo guarda en un archivo; imprime solo la parte pública	Tener la clave de la entidad
<code>jq -r 'keys[]' \$W/c005-clave.json</code>	Lista los nombres de los campos del archivo (no sus valores)	Saber qué contiene sin mostrar secretos

✓ **Lectura. Respuesta a P1.** El archivo tiene **dos campos**: `privateKeyPkcs8` (la clave **privada**, que nunca debe salir de la entidad) y `publicKeyMultibase` (la pública, la que irá en el documento). El comando solo muestra los *nombres*, no los valores: **nunca se imprime la clave privada**.

17.2 Construir el documento

```
$ T00LS build-doc --did $DID --key c005-clave.json --out c005-doc.json >/dev/null; jq . $W/c005-doc.json
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/multikey/v1"
  ],
  "id": "did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319",
  "verificationMethod": [
    {
      "id": "did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319#key-1",
      "type": "Multikey",
      "controller": "did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319",
      "publicKeyMultibase": "zDnaemzHWpsCu6cc4V5Er6reddfTqoGRiypwqp2mJgX4WHDfh"
    }
  ]
}
```

```

"authentication": [
  "did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319#key-1"
],
"assertionMethod": [
  "did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319#key-1"
]
}
$ jq -c 'keys' $W/c005-doc.json
["@context","assertionMethod","authentication","id","verificationMethod"]
$ grep -c -iE "privateKey|pkcs8" $W/c005-doc.json
0

```

Comando	Qué hace	Qué se busca
TTOOLS build-doc --did \$DID --key c005-clave.json --out c005-doc.json	Arma el DID Document con la clave pública del archivo	El documento para publicar
jq . \$W/c005-doc.json	Muestra el documento con formato legible	Verlo completo
jq -c 'keys' ...	Lista los nombres de sus propiedades	Ver exactamente qué lleva
grep -c -iE "privateKey pkcs8" ...	Cuenta cuántas líneas mencionan esas palabras	Comprobar que no hay secreto

✓ Lectura. Respuestas a P2 y P3.

- **P2:** cinco propiedades: `@context`, `id`, `verificationMethod`, `authentication` y `assertionMethod`. No lleva `service` porque es opcional.
- **P3:** el conteo es **0** (`grep -c` termina con código 1 cuando cuenta cero; eso es lo que queremos).
- Fíjate en la clave dentro del documento: `"type": "Multikey"`, con el `controller` igual al propio DID, y en que `authentication` y `assertionMethod` **apuntan a esa misma clave** (`...#key-1`): "esta clave sirve para identificarse y para firmar afirmaciones".
- Observa que el orden de campos aquí es el del constructor. Cuando el registro lo guarde y lo sirva, lo escribirá **ordenado alfabéticamente y sin espacios** (JSON canónico, §13.3); lo verás en §18.

17.3 Qué hay dentro de un Multikey

```

$ python3 - <<'EOF'
> import json
> A = "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"
> k = json.load(open("../evidencias/work/c005-clave.json"))["publicKeyMultibase"]
> n = 0
> for ch in k[1:]: n = n * 58 + A.index(ch)
> b = n.to_bytes(35, "big")
> print("texto          :", k[1:14] + "...   (la 'z' inicial significa base58btc)")
> print("bytes decodificados:", len(b))
> print("prefijo (tipo)   :", b[2:].hex(), " <- 0x8024 = clave pública P-256")
> print("punto comprimido :", b[2:3].hex(), "+ 32 bytes de la coordenada x   (02 o 03 = paridad de y)")
> EOF
texto          : zDnaemzHWpsCu6...   (la 'z' inicial significa base58btc)
bytes decodificados: 35
prefijo (tipo)   : 8024 <- 0x8024 = clave pública P-256
punto comprimido : 03 + 32 bytes de la coordenada x   (02 o 03 = paridad de y)

```

Parte de la salida	Qué significa
zDn... (la z)	El texto está en base58btc
bytes decodificados: 35	Al decodificarlo salen 35 bytes
prefijo 8024	Dos bytes que dicen "esto es una clave pública P-256 " (código <i>multicodec</i> 0x1200 escrito en forma compacta)
02 / 03 + 32 bytes	El punto comprimido de la clave: un byte de paridad y la coordenada x

✓ **Lectura. Respuesta a P4.** zDn... no es un texto aleatorio: es **una etiqueta (el prefijo 8024) más la clave** en 33 bytes. Por eso **todas** las claves P-256 en Multikey empiezan por zDn .

🎯 Conclusión del laboratorio B

Lo que pide el PDF	Evidencia
Clave P-256 en Multikey	type: Multikey , texto zDn... que decodifica a prefijo 8024 + punto comprimido (§17.3)
Solo material público	Cinco propiedades; 0 menciones de claves privadas (§17.2)
La clave privada queda fuera	Vive en el archivo de claves; el documento solo recibe la pública (§17.1)
Prueba automática equivalente	Erso005DidWebTest > criterio 3 - el documento publicado solo contiene material publico (§22)

18. CRITERIO 1 — El documento se sirve por HTTPS en la URL calculada

"El documento se sirve por HTTPS en la URL calculada; evidencia: lectura desde la URL pública."

Objetivo. Publicar el documento y leerlo por la **puerta pública**, exactamente como lo haría cualquier tercero, comprobando que responde bien, con el tipo correcto, y que las vías indebidas están cerradas.

Qué se va a demostrar (afirmación comprobable):

1. La publicación se acepta y se **confirma** (el propio registro lee de vuelta lo publicado).
2. Un GET a la URL calculada devuelve 200 con application/did+json .
3. El contenido servido es el documento, ya en forma canónica.
4. Otras rutas, otros verbos y DID inexistentes **no** lo devuelven.
5. El consumidor de DID (otra ERSO) lo **resuelve**.

🔧 Predice antes de ejecutar

- **P1.** La publicación, ¿qué código devuelve y en qué estado queda?
- **P2.** La lectura por la puerta pública, ¿qué código y qué tipo de contenido tendrá?
- **P3.** ¿Quién agrega los encabezados ETag , X-Content-Hash y nosniff : la aplicación o nginx?

- **P4.** ¿Qué código dará un POST sobre el `did.json`? ¿Y pedir `index.html`? ¿Y un DID que no existe?

18.1 Publicar

```
$ PUBLICAR $DID --purpose CREATE --expected 0 --key c005-clave.json --doc c005-doc.json
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=a6326d6f-72ed-4cca-b8ca-6be54ca06c0d audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=CREATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319#key-
1 docHash=sha256:96c1fa022d9f33f2035f738730e40d555503b322d59fe66dc863fec542d68706
[4/4] escritura .... 201 If-Match=0 Idempotency-Key=bb6f6cf9-25dc-470e-9a90-5cd6d6e96484
{
  "operationId": "21523af0-c2b3-4dd1-aed5-6a9f2db7ee85",
  "did": "did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319",
  "purpose": "CREATE",
  "status": "CONFIRMED",
  "version": 1,
  "hash": "sha256:96c1fa022d9f33f2035f738730e40d555503b322d59fe66dc863fec542d68706",
  "publicUrl": "https://civica-desarrollo.avance.org.co/lab/prueba-005-1790872319/did.json"
}
```

Parte de la salida	Qué significa
[1/4] token ... OK (client_credentials + mTLS)	La entidad se autenticó: certificado cliente y secreto
[2/4] desafío ... 201 ... purpose=CREATE	El servidor entregó un desafío de un solo uso para crear
[3/4] prueba ... ES256 kid=...#key-1 docHash=...	La entidad firmó el desafío y el hash del documento con su clave privada (prueba de posesión)
[4/4] escritura ... 201 If-Match=0	Se creó (201); If-Match: 0 significa "espero que no exista"
status: CONFIRMED, version: 1, hash, publicUrl	El registro leyó de vuelta la URL pública y el hash coincidió

✓ **Lectura. Respuesta a P1:** 201, estado CONFIRMED, versión 1. La publicUrl es la misma que calculó el laboratorio A.

18.2 Leer por la puerta pública (los encabezados)

```
$ curl -s -D - -o /dev/null $PUB https://$D:8443/lab/$ID/did.json
HTTP/1.1 200 OK
Server: nginx
Date: Thu, 01 Oct 2026 16:32:09 GMT
Content-Type: application/did+json
Content-Length: 621
Connection: keep-alive
ETag: "1"
Cache-Control: no-cache
X-Content-Hash: sha256:96c1fa022d9f33f2035f738730e40d555503b322d59fe66dc863fec542d68706
X-Content-Type-Options: nosniff
```

Comando	Qué hace	Qué se busca
curl -s -D - -o /dev/null \$PUB https://\$D:8443/lab/\$ID/did.json	-D - imprime los encabezados de la respuesta; -o /dev/null descarta el cuerpo	Ver cómo responde el servidor

Encabezado	Qué dice
HTTP/1.1 200 OK	La lectura funcionó
Content-Type: application/did+json	Es un DID Document
ETag: "1"	Es la versión 1
Cache-Control: no-cache	Antes de reutilizar una copia, pregunta si sigue vigente
X-Content-Hash: sha256:...	La huella del contenido servido
X-Content-Type-Options: nosniff	El navegador no debe adivinar el tipo

✓ **Lectura. Respuestas a P2 y P3.** 200 y application/did+json. ETag, Cache-Control y X-Content-Hash los agrega la **aplicación**; nosniff lo agrega **nginx** (aparece en todas las respuestas de esa puerta).

18.3 El contenido

```
$ curl -s $PUB https://$D:8443/lab/$ID/did.json | jq .
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/multikey/v1"
  ],
  "assertionMethod": [
    "did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319#key-1"
  ],
  "authentication": [
    "did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319#key-1"
  ],
  "id": "did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319",
  "verificationMethod": [
    {
      "controller": "did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319",
      "id": "did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319#key-1",
      "publicKeyMultibase": "zDnaemzHWpsCu6cc4V5Er6reddfTqoGRiypwqp2mJgX4WHDfh",
      "type": "Multikey"
    }
  ]
}
```

✓ **Lectura.** Es el documento de §17, pero ahora en forma **canónica**: los campos aparecen **ordenados alfabéticamente** (@context, assertionMethod, authentication, id, verificationMethod) y las propiedades de la clave también (controller, id, publicKeyMultibase, type). Es la forma que se guarda, se firma y se sirve. (jq . solo la muestra con sangría para leerla.)

18.4 Las vías indebidas están cerradas

```
$ curl -s -o /dev/null -w "otra ruta (index.html)    -> HTTP %{http_code}\n" $PUB
https://$D:8443/lab/$ID/index.html
otra ruta (index.html)    -> HTTP 404
$ curl -s -o /dev/null -w "POST sobre did.json      -> HTTP %{http_code}\n" -X POST $PUB
https://$D:8443/lab/$ID/did.json
POST sobre did.json      -> HTTP 403
$ curl -s -o /dev/null -w "DID que no existe        -> HTTP %{http_code}\n" $PUB
https://$D:8443/lab/no-existe-$ID/did.json
DID que no existe        -> HTTP 404
```

Comando	Qué hace	Qué se busca
<code>curl ... /lab/\$ID/index.html</code>	Pide otro archivo de la misma carpeta	404 : nada más que <code>did.json</code> se sirve
<code>curl -X POST ... did.json</code>	Intenta enviar algo por la puerta pública	403 : no se escribe por aquí
<code>curl ... /lab/no-existe-\$ID/did.json</code>	Pide el documento de un DID que no existe	404

✓ **Lectura. Respuesta a P4:** 404, 403 y 404. Solo GET de un `did.json` existente devuelve contenido.

18.5 El consumidor de DID lo resuelve

```
$ T00LS resolve --did $DID --ca /certs/ca.crt | grep -E "^(resolutionMetadata|documentMetadata)" | cut -c1-210
resolutionMetadata: ResolutionMetadata(error=null, message=null, contentType=application/did+json, url=https://civica-desarrollo.avance.org.co/lab/prueba-005-1790872319/did.json, violations=[])
documentMetadata: DocumentMetadata(deactivated=false, versionId=1, contentHash=sha256:96c1fa022d9f33f2035f738730e40d555503b322d59fe66dc863fec542d68706)
```

Comando	Qué hace	Qué se busca
<code>T00LS resolve --did \$DID --ca /certs/ca.crt</code>	Usa el consumidor de DID (otra ERSO): convierte el DID en URL, lo descarga por HTTPS, valida su estructura	Un tercero, independiente del registro, lo acepta
<code> grep -E "^(resolutionMetadata documentMetadata)"</code>	Se queda con las dos líneas de metadatos	Ver el resultado

✓ **Lectura.** `error=null`, `contentType=application/did+json` y la `url` coincide con la calculada. En `documentMetadata: deactivated=false, versionId=1` y el `contentHash`. Es decir, **un cliente que no conoce el registro** pudo encontrar y validar el documento usando solo el DID.

🎯 Conclusión — evidencia del criterio 1

Lo que pide el criterio	Evidencia en este laboratorio
Se sirve por HTTPS	GET por TLS → 200 (§18.2)
En la URL calculada	La <code>publicUrl</code> coincide con la del paso 1; el consumidor resuelve el DID por esa URL (§18.1 y §18.5)
Tipo correcto	Content-Type: <code>application/did+json</code> (§18.2)
Lectura desde la URL pública	Todo se hizo por la puerta pública, sin credenciales (§18.2 a §18.5)
Casos negativos	POST → 403, otra ruta → 404, DID inexistente → 404 (§18.4)
Prueba automática equivalente	Erso005DidWebTest > criterio 1 y 2 - se sirve en la URL calculada... (§22)

19. CRITERIO 2 — El documento contiene la clave pública correcta

"El documento contiene la clave pública correcta; evidencia: documento publicado y comparación de hash."

Objetivo. Demostrar de **tres maneras independientes** que la clave publicada es la correcta y que el contenido servido es el que se escribió.

Qué se va a demostrar:

1. El hash del contenido **servido**, el que **informa** el servidor y el que **registró** el registro son idénticos.
2. La clave publicada es **idéntica** a la que se generó.
3. La clave publicada **verifica firmas** de nuestra clave privada (prueba funcional).
4. Cambiar un carácter cambia el hash.

Predice antes de ejecutar

- **P1.** Los tres hashes (servido, encabezado, registrado), ¿serán iguales?
- **P2.** La clave generada y la publicada, ¿serán idénticas?
- **P3.** Comparar textos prueba que son iguales, pero ¿cómo probarías que la clave publicada **sirve** para verificar lo que firma nuestra clave privada?
- **P4.** Si alguien altera un carácter del documento servido, ¿qué pasa con el hash?

19.1 Tres huellas del mismo contenido

```
$ curl -s $PUB https://$D:8443/lab/$ID/did.json | sha256sum | awk '{print "hash del contenido servido : sha256:"$1}'
hash del contenido servido      :
sha256:96c1fa022d9f33f2035f738730e40d555503b322d59fe66dc863fec542d68706
$ curl -s -D - -o /dev/null $PUB https://$D:8443/lab/$ID/did.json | grep -i "x-content-hash" | sed 's/^[^:]*: /hash que informa el servidor : /'
hash que informa el servidor    :
sha256:96c1fa022d9f33f2035f738730e40d555503b322d59fe66dc863fec542d68706
$ LABOP --path "/admin/v1/documents/$DID/versions" 2>&1 | sed -n '/^[/,,$p' | jq -r '"hash que registró el registro : " + .[-1].hash'
hash que registró el registro  :
sha256:96c1fa022d9f33f2035f738730e40d555503b322d59fe66dc863fec542d68706
```

Comando	Qué hace	Qué se busca
<code>curl ... did.json sha256sum</code>	Descarga el documento y calcula su huella con una herramienta propia del sistema operativo	La huella de lo que ve el mundo
<code>curl -D - ... grep -i x-content-hash</code>	Lee la huella que declara el servidor en el encabezado	La huella que dice haber servido
<code>LABOP ... versions jq[-1].hash</code>	Pide al registro el hash de la última versión guardada	La huella que el registro dice haber escrito

 **Lectura. Respuesta a P1.** Las **tres** huellas son idénticas. Tres fuentes independientes (nuestro cálculo, lo que declara el servidor y lo que guardó el registro) coinciden: lo que

cualquiera lee es exactamente lo que se escribió.

19.2 La clave publicada es la que se generó

```
$ GEN=$(jq -r .publicKeyMultibase $W/c005-clave.json); echo "clave pública generada : $GEN"
clave pública generada : zDnaemzHWpsCu6cc4V5Er6reddfTqoGRiypwqp2mJgX4WHDfh
$ PUBK=$(curl -s $PUB https://$D:8443/lab/$ID/did.json | jq -r
'.verificationMethod[0].publicKeyMultibase'); echo "clave pública publicada : $PUBK"
clave pública publicada : zDnaemzHWpsCu6cc4V5Er6reddfTqoGRiypwqp2mJgX4WHDfh
$ [ "$GEN" = "$PUBK" ] && echo "RESULTADO: las dos claves son IDÉNTICAS" || echo "RESULTADO: NO
coinciden"
RESULTADO: las dos claves son IDÉNTICAS
```

Comando	Qué hace
<code>jq -r .publicKeyMultibase \$W/c005-clave.json</code>	Lee la clave pública del archivo que generó la entidad
<code>curl ... did.json jq -r '.verificationMethod[0].publicKeyMultibase'</code>	Lee la clave pública del documento publicado
<code>["\$GEN" = "\$PUBK"] && ...</code>	Compara los dos textos

✓ **Lectura. Respuesta a P2.** las dos claves son IDÉNTICAS.

19.3 La clave publicada realmente funciona

💡 Que dos textos sean iguales no demuestra que la clave **sirva**. La prueba de fondo es esta: firmar con nuestra clave **privada** y comprobar que alguien, usando **solo lo publicado**, valida esa firma.

```
$ JWS=$(TOOLS sign --key c005-clave.json --kid "$DID#key-1" --message "prueba-005" | tail -1); echo
"${JWS:0:60}..."
eyJhbGciOiJIUzI1NiIsImtpZCI6ImRpZDp3ZWl6Y2l2aWVhLWRLc2Fycm9s...
$ TOOLS verify --jws "$JWS" --purpose assertionMethod --did $DID --ca /certs/ca.crt
VALIDA did=did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319 kid=did:web:civica-
desarrollo.avance.org.co:lab:prueba-005-1790872319#key-1 payload=prueba-005
```

Comando	Qué hace	Qué se busca
<code>TOOLS sign --key c005-clave.json - -kid "\$DID#key-1" --message "prueba-005"</code>	Firma un mensaje con la clave privada y genera un JWS (se muestran solo los primeros 60 caracteres)	Una firma real hecha con nuestra clave
<code>TOOLS verify --jws "\$JWS" -- purpose assertionMethod --did \$DID --ca ...</code>	El consumidor resuelve el DID por HTTPS , toma la clave publicada y valida la firma	Que la clave publicada sea la pareja de nuestra privada

✓ **Lectura. Respuesta a P3.** Responde `VALIDA`, con el `DID`, el `kid` y el `payload`. La clave publicada **verificó una firma hecha con nuestra privada**: no solo es igual como texto, es **funcionalmente la correcta**. Y `--purpose assertionMethod` comprobó además que la clave está **autorizada** para firmar afirmaciones.

19.4 Cambiar un carácter cambia el hash

```
$ curl -s $PUB https://$D:8443/lab/$ID/did.json | sed 's/zDn/zDm/' | sha256sum | awk '{print "hash si se altera un carácter : sha256:"$1}'  
hash si se altera un carácter :  
sha256:facfb071668925502eedc7761fbbcd0b53606503bd4491da495f76430020b7fc
```

Comando	Qué hace
<code>... sed 's/zDn/zDm/' sha256sum</code>	Cambia una letra (<code>n</code> por <code>m</code>) de la clave en el texto descargado y calcula la huella de esa copia alterada

✅ **Lectura. Respuesta a P4.** La huella es **completamente distinta** de las tres anteriores (efecto avalancha). Por eso comparar hashes sirve para detectar cualquier alteración, por pequeña que sea.

🎯 Conclusión — evidencia del criterio 2

Lo que pide el criterio	Evidencia en este laboratorio
Documento publicado	El <code>did.json</code> servido (§18.3)
Comparación de hash	Tres huellas idénticas: servida, declarada y registrada (§19.1)
Clave pública correcta	Idéntica a la generada (§19.2) y verifica firmas de la privada (§19.3)
Sensibilidad de la comprobación	Un carácter alterado cambia el hash (§19.4)
Prueba automática equivalente	Erso005DidWebTest > criterio 1 y 2 - ...con la clave publica correcta y hash igual (§22)

20. CRITERIO 3 — El documento no expone claves privadas ni datos civiles

"El documento no expone claves privadas ni datos civiles; evidencia: revisión del contenido publicado."

Objetivo. Revisar el contenido publicado **y** demostrar que un documento con material prohibido **no llega a publicarse**.

💡 **Por qué hay dos partes.** "No contiene secretos" no se prueba mirando un documento bueno: hay que **fabricar documentos malos** y comprobar que ninguno entra. Sin eso, lo que se tiene es una demostración, no una prueba.

🔧 Predice antes de ejecutar

- **P1.** ¿Qué propiedades tiene el documento publicado?
- **P2.** Si buscas en él palabras como `private`, `secret`, `credentialSubject`, `birthDate` o `"d":`, ¿cuántas coincidencias hay?
- **P3.** Si buscas la **clave privada real** (el contenido del archivo de claves), ¿cuántas coincidencias hay?

- **P4.** Se intenta publicar cinco documentos con material prohibido: una clave privada, un dato civil, otro dato civil, una propiedad desconocida y un servicio `http`. ¿Se publica alguno? ¿Qué código y qué motivo devuelve cada uno?

20.1 Revisar el contenido publicado

```
$ curl -s $PUB https://$D:8443/lab/$ID/did.json | jq -c 'keys'
["@context","assertionMethod","authentication","id","verificationMethod"]
$ curl -s $PUB https://$D:8443/lab/$ID/did.json | grep -c -iE
'private|secret|credentialSubject|birthDate|"d":'
0
$ PRIV=$(jq -r .privateKeyPkcs8 $W/c005-clave.json); curl -s $PUB https://$D:8443/lab/$ID/did.json |
grep -c -F "$PRIV"
0
```

Comando	Qué hace	Qué se busca
<code>jq -c 'keys'</code>	Lista las propiedades del documento publicado	Solo las autorizadas
<code>grep -c -iE 'private secret credentialSubject birthDate "d":'</code>	Cuenta las líneas que contienen alguna de esas palabras (sin distinguir mayúsculas)	0
<code>PRIV=\$(jq -r .privateKeyPkcs8 ...); ... grep -c -F "\$PRIV"</code>	Toma la clave privada real del archivo y la busca, texto exacto , dentro del documento (<code>-F</code> = texto literal)	0 : nunca aparece

✅ **Lectura. Respuestas a P1, P2 y P3.** Cinco propiedades, todas autorizadas. Las dos búsquedas devuelven `0` (el código de salida 1 de `grep -c` significa "ninguna coincidencia", que es lo que queremos). El tercer comando es el más fuerte: no busca *palabras*, busca **el valor secreto real**, y no está. 💡 *El valor del secreto nunca se muestra en pantalla: solo se usa para buscar.*

20.2 Intentar publicar documentos con material prohibido

Partimos de un documento válido para **otro DID** (`...-neg`) y le agregamos, uno a uno, campos que no deberían aceptarse:

```
$ DIDNEG=did:web:$D:lab:$ID-neg; TOOLS build-doc --did $DIDNEG --key c005-clave.json --out c005-neg.json >/dev/null; echo "documento base para los intentos: $DIDNEG"
documento base para los intentos: did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319-neg
```

Intento 1 — una clave privada dentro de la clave pública

```
$ jq '.verificationMethod[0].privateKeyMultibase="z1234"' $W/c005-neg.json > $W/c005-neg1.json;
PUBLICAR $DIDNEG --purpose CREATE --expected 0 --key c005-clave.json --doc c005-neg1.json 2>&1 | tail
-n +4
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=ed0ad325-ba13-4e82-a9e1-fa05a07e6cd0
{
  "error": "INVALID_DOCUMENT",
  "message": "El documento no cumple el perfil de publicación",
  "details": [
    "PRIVATE_KEY_MATERIAL: Campo prohibido con posible clave privada en
```

```
$ verificationMethod[0].privateKeyMultibase"  
  ]  
}
```

Intento 2 — un dato civil (credentialSubject con un nombre)

```
$ jq '.credentialSubject={"name":"Ana Pérez"}' $W/c005-neg.json > $W/c005-neg2.json; PUBLICAR $DIDNEG  
--purpose CREATE --expected 0 --key c005-clave.json --doc c005-neg2.json 2>&1 | tail -n +4  
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=16a9f748-eb8d-4eb3-a23f-7b2a00401bed  
{  
  "error": "INVALID_DOCUMENT",  
  "message": "El documento no cumple el perfil de publicación",  
  "details": [  
    "CIVIL_DATA: Campo de datos civiles no permitido en $.credentialSubject",  
    "CIVIL_DATA: Campo de datos civiles no permitido en $.credentialSubject.name",  
    "UNKNOWN_PROPERTY: Propiedad no permitida en el perfil de publicación: 'credentialSubject'"  
  ]  
}
```

Intento 3 — otro dato civil (birthDate)

```
$ jq '.birthDate="1990-01-01"' $W/c005-neg.json > $W/c005-neg3.json; PUBLICAR $DIDNEG --purpose  
CREATE --expected 0 --key c005-clave.json --doc c005-neg3.json 2>&1 | tail -n +4  
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=fe58221c-5b3f-4c9e-a5df-a817cb7bc329  
{  
  "error": "INVALID_DOCUMENT",  
  "message": "El documento no cumple el perfil de publicación",  
  "details": [  
    "CIVIL_DATA: Campo de datos civiles no permitido en $.birthDate",  
    "UNKNOWN_PROPERTY: Propiedad no permitida en el perfil de publicación: 'birthDate'"  
  ]  
}
```

Intento 4 — una propiedad desconocida (extension)

```
$ jq '.extension="valor propietario"' $W/c005-neg.json > $W/c005-neg4.json; PUBLICAR $DIDNEG --  
purpose CREATE --expected 0 --key c005-clave.json --doc c005-neg4.json 2>&1 | tail -n +4  
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=52bed5d2-fa7d-4316-9638-8dd9f43854f7  
{  
  "error": "INVALID_DOCUMENT",  
  "message": "El documento no cumple el perfil de publicación",  
  "details": [  
    "UNKNOWN_PROPERTY: Propiedad no permitida en el perfil de publicación: 'extension'"  
  ]  
}
```

Intento 5 — un servicio con dirección http (no segura)

```
$ jq '.service=[{"id":"#s","type":"X","serviceEndpoint":"http://inseguro.example"}]' $W/c005-neg.json  
> $W/c005-neg5.json; PUBLICAR $DIDNEG --purpose CREATE --expected 0 --key c005-clave.json --doc c005-  
neg5.json 2>&1 | tail -n +4  
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=9a6fd278-a91d-4e29-9f3f-ff64c04930ba  
{  
  "error": "INVALID_DOCUMENT",  
  "message": "El documento no cumple el perfil de publicación",  
  "details": [  
    "INVALID_SERVICE: service[0].serviceEndpoint debe ser una URL https"  
  ]  
}
```


Comando	Qué hace	Qué se busca
<code>jq '.verificationMethod[0].privateKeyMultibase="z1234"' ... > c005-neg1.json</code>	Fabrica una copia del documento a la que le añade un campo prohibido	El documento "malo"
<code>PUBLICAR \$DIDNEG --purpose CREATE --expected 0 ... --doc c005-neg1.json</code>	Intenta publicarlo con la entidad legítima	Que el registro lo rechace
<code> tail -n +4</code>	Muestra desde la línea [4/4] (la escritura) hasta el final	Ver el código HTTP y el motivo

Intento	Código	Motivo devuelto
1 · clave privada	422	PRIVATE_KEY_MATERIAL: campo prohibido, y dónde está (\$.verificationMethod[0].privateKeyMultibase)
2 · credentialSubject	422	CIVIL_DATA (el campo y el subcampo name) y UNKNOWN_PROPERTY
3 · birthDate	422	CIVIL_DATA y UNKNOWN_PROPERTY
4 · extension	422	UNKNOWN_PROPERTY
5 · servicio http	422	INVALID_SERVICE: el serviceEndpoint debe ser https

✅ **Lectura. Respuesta a P4. Ninguno se publica:** los cinco devuelven 422 INVALID_DOCUMENT con el motivo exacto. Fíjate en dos cosas: (a) el campo prohibido se detecta **en cualquier profundidad** (estaba dentro de la clave); (b) los datos civiles fallan **dos veces**: por ser datos civiles y por no estar en la lista blanca. Esa doble red es deliberada: si una regla se olvidara, la otra lo atraparía.

20.3 Confirmar que no quedó nada

```
$ echo "DID registrados con ese identificador: $($PSQL -tAc "select count(*) from did_documents where did = '$DIDNEG'"); curl -s -o /dev/null -w "lectura pública de $ID-neg -> HTTP %{http_code}\n" $PUB https://$D:8443/lab/$ID-neg/did.json
DID registrados con ese identificador: 0
lectura pública de prueba-005-1790872319-neg -> HTTP 404
```

Comando	Qué hace
<code>\$PSQL -tAc "select count(*) from did_documents where did = '\$DIDNEG'"</code>	Cuenta en la base de datos cuántos DID tienen ese identificador
<code>curl ... /lab/\$ID-neg/did.json</code>	Intenta leerlo por la puerta pública

✅ **Lectura.** 0 registrados y HTTP 404: **nada quedó guardado ni publicado** de los cinco intentos. El rechazo ocurre **antes** de escribir.

🎯 Conclusión — evidencia del criterio 3

Lo que pide el criterio	Evidencia en este laboratorio
No expone claves privadas	0 coincidencias con palabras de clave privada y 0 con el valor real de la clave privada (\$20.1)

Lo que pide el criterio	Evidencia en este laboratorio
No expone datos civiles	Solo cinco propiedades autorizadas (§20.1)
Revisión del contenido publicado	Las tres revisiones de §20.1
Caso negativo	Cinco documentos con material prohibido → 422 , ninguno se guarda (§20.2 y §20.3)
Prueba automática equivalente	Erso005DidWebTest > criterio 3 - el documento publicado solo contiene material publico (§22)

21. CRITERIO 4 — Un `id` desajustado se rechaza y el camino base opera con la extensión desactivada

"Un `id` desajustado se rechaza y el camino base opera con la extensión desactivada; evidencia: pruebas negativas y de regresión."

Objetivo. Demostrar las **dos mitades** del criterio: (a) el rechazo de documentos cuyo `id` no corresponde al DID donde se publican, y (b) la regresión con la extensión apagada.

💡 **Por qué importa el `id`.** El `id` del documento debe ser **el DID que lo contiene**. Si se pudiera publicar, en la ruta de A, un documento que dice ser B, alguien podría hacerse pasar por B desde la dirección de A.

🔧 Predice antes de ejecutar

- **P1.** Se intenta publicar, en la ruta de un DID, un documento cuyo `id` es de **otro** DID. ¿Qué responde el registro? ¿Queda algo guardado?
- **P2.** Se intenta publicar un DID de **ciudadano** (`...:ciudadanos:ana`), uno de un **dominio ajeno** y uno con una ruta `...`. ¿Qué pasa en cada caso y **en qué momento** (antes o después de pedir el desafío)?
- **P3.** ¿Cuántos DID de ciudadanos hay publicados en el registro?
- **P4.** Con la extensión apagada, ¿qué responden `/health`, `/base/ping` y el `did.json`?

21.1 Parte (a): el `id` desajustado

```
$ DIDOTRO=did:web:$D:lab:$ID-otro; TOOLS build-doc --did $DIDOTRO --key c005-clave.json --out c005-otro.json >/dev/null; jq -r .id $W/c005-otro.json
did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319-otro
$ PUBLICAR $DIDNEG --purpose CREATE --expected 0 --key c005-clave.json --doc c005-otro.json 2>&1 | tail -n +4
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=37856b34-85b4-4207-b75e-d82914e605c1
{
  "error": "INVALID_DOCUMENT",
  "message": "El documento no cumple el perfil de publicación",
  "details": [
    "ID_MISMATCH: El id 'did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319-otro' no coincide con el DID esperado 'did:web:civica-desarrollo.avance.org.co:lab:prueba-005-1790872319-neg'"
  ]
}
```

Comando	Qué hace	Qué se busca
<code>TTOOLS build-doc --did \$DIDOTRO ... --out c005-otro.json</code>	Arma un documento cuyo <code>id</code> es <code>...-otro</code>	Un documento que "dice ser otro"
<code>PUBLICAR \$DIDNEG ... --doc c005-otro.json</code>	Intenta publicarlo en la ruta de <code>...-neg</code>	Que se rechace

✓ **Lectura. Respuesta a P1.** 422 INVALID_DOCUMENT con ID_MISMATCH: "El id '...-otro' no coincide con el DID esperado '...-neg'". El servidor compara el `id` del documento con el DID de la ruta **antes** de aceptar nada.

```
$ echo "DID registrados bajo $ID-neg: $($PSQL -tAc "select count(*) from did_documents where did like '%$ID-neg'"); curl -s -o /dev/null -w "lectura pública -> HTTP %{http_code}\n" $PUB
https://$D:8443/lab/$ID-neg/did.json
DID registrados bajo prueba-005-1790872319-neg: 0
lectura pública -> HTTP 404
```

✓ **Lectura.** 0 registrados y 404 en la lectura pública: **no quedó nada**.

21.2 Parte (a), variantes: DID que no deben poder publicarse

```
$ PUBLICAR did:web:$D:ciudadanos:ana --purpose CREATE --expected 0 --key c005-clave.json --doc c005-doc.json 2>&1 | tail -3
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío FALLÓ: HTTP 412 {"error":"PRECONDITION_FAILED","message":"No se emite desafío: precondiciones incumplidas","details":["NAMESPACE_NOT_RESERVED"]}
$ PUBLICAR did:web:otro.example:lab:x --purpose CREATE --expected 0 --key c005-clave.json --doc c005-doc.json 2>&1 | tail -3
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío FALLÓ: HTTP 412 {"error":"PRECONDITION_FAILED","message":"No se emite desafío: precondiciones incumplidas","details":["DID_DOMAIN_MISMATCH"]}
$ PUBLICAR "did:web:$D:lab:.." --purpose CREATE --expected 0 --key c005-clave.json --doc c005-doc.json 2>&1 | tail -3
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío FALLÓ: HTTP 412 {"error":"PRECONDITION_FAILED","message":"No se emite desafío: precondiciones incumplidas","details":["DID_INVALID"]}
$ echo "DID de ciudadanos publicados (ruta /ciudadanos/): $($PSQL -tAc "select count(*) from did_documents where did like ':%ciudadanos:%')"
```

Intento	Qué se probó	Respuesta	Qué significa
DID de ciudadano	<code>did:web:\$D:ciudadanos:ana</code>	412 NAMESPACE_NOT_RESERVED	Esa ruta no está reservada a ninguna entidad ; nadie puede crearla (condición 3)
Dominio ajeno	<code>did:web:otro.example:lab:x</code>	412 DID_DOMAIN_MISMATCH	Solo se publican DID del dominio configurado
Ruta con ..	<code>did:web:\$D:lab:..</code>	412 DID_INVALID	El servidor repite la validación de segmentos del laboratorio A
Conteo	DID con <code>:ciudadanos:</code> en la base	0	No se publica por defecto ningún DID de ciudadano

✓ **Lectura. Respuestas a P2 y P3.** En los tres casos el rechazo es **412 en el paso [2/4]**, es decir, **antes de entregar el desafío**: el registro verifica las precondiciones primero y ni siquiera inicia el trámite. Y el conteo de DID de ciudadanos es 0.

21.3 Parte (b): regresión con la extensión apagada

```
$ docker rm -f vdr-off >/dev/null 2>&1; docker run -d --name vdr-off -p 18080:8080 vdr-ssi/vdr-  
service:local >/dev/null; sleep 6  
$ curl -s http://127.0.0.1:18080/health  
{"status":"UP","vdr":"disabled"}  
$ curl -s http://127.0.0.1:18080/base/ping  
pong  
$ curl -s -o /dev/null -w "did.json con la extensión apagada -> HTTP %{http_code}\n"  
http://127.0.0.1:18080/lab/laboratorio/did.json  
did.json con la extensión apagada -> HTTP 404  
$ docker rm -f vdr-off  
vdr-off
```

Comando	Qué hace	Qué se busca
<code>docker run -d --name vdr-off -p 18080:8080 ...</code>	Arranca una copia temporal del servicio sin definir VDR_ENABLED (queda apagada por defecto) y publicada en el puerto 18080	El comportamiento sin la extensión
<code>curl ... /health y /base/ping</code>	Consulta el camino base	Que sigan respondiendo
<code>curl ... /lab/laboratorio/did.json</code>	Pide un documento del registro	Que no exista esa ruta
<code>docker rm -f vdr-off</code>	Borra la copia temporal	Dejar limpio

✓ **Lectura. Respuesta a P4.** `{"status":"UP","vdr":"disabled"}` (sano, con la extensión apagada), `pong`, y `404` en el `did.json`. El camino base **opera**; la extensión **no existe**. (El detalle completo de esta regresión —sin base de datos, con el canal de escritura cerrado— está en el informe de la ERSO 004, criterio 3.)

🎯 Conclusión — evidencia del criterio 4

Lo que pide el criterio	Evidencia en este laboratorio
Un <code>id</code> desajustado se rechaza	<code>422 ID_MISMATCH</code> , nada guardado, <code>404</code> público (§21.1)
Pruebas negativas	DID de ciudadano, dominio ajeno y segmento <code>..</code> rechazados con <code>412</code> antes del desafío (§21.2)
El camino base opera con la extensión desactivada	<code>UP/disabled</code> , <code>pong</code> ; <code>did.json</code> → <code>404</code> (§21.3)
Prueba de regresión	Comportamiento del camino base idéntico, solo cambia <code>enabled/disabled</code>
Prueba automática equivalente	<code>Erso005DidWebTest > criterio 4 - un id desajustado se rechaza y no se publica por defecto ningún DID de ciudadano...</code> (§22)

22. Pruebas automáticas y recorrido de extremo a extremo

Los laboratorios anteriores son **pruebas manuales** (las haces tú, viendo cada respuesta). Además existen **pruebas automáticas**: programas que repiten las mismas verificaciones en segundos y que se ejecutan cada vez que cambia el código.

22.1 Ejecutar las pruebas de la ERSO 005

 **Requisito.** Una base de datos desechable para las pruebas (vdr-test-pg , puerto 55432); **no toca el proyecto levantado.**

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect
$ TEST_DB_URL=jdbc:postgresql://127.0.0.1:55432/wallet_test ./gradlew :vdr-service:test --tests '*Erso005*' --rerun-tasks -q 2>&1 | tail -3; echo "(sin salida = todas pasaron)"
(sin salida = todas pasaron)
$ python3 - <<'EOF'
> import re, glob, html
> f = glob.glob('vdr-service/build/test-results/test/*Erso005*.xml')[0]
> s = open(f, encoding='utf-8').read()
> tot = re.search(r'tests="(\d+)" skipped="(\d+)" failures="(\d+)"', s).groups()
> print(f"resultado: {tot[0]} pruebas, {tot[1]} omitidas, {tot[2]} fallos\n")
> for m in re.finditer(r'<testcase name="([\^"]+)"[\^>]*?(</>|>(.*)</testcase>)', s, re.S):
>     name = html.unescape(m.group(1)).removesuffix("(")
>     estado = "FALLA" if "<failure" in (m.group(3) or "") else "PASA "
>     print(f" [{estado}] {name}")
> EOF
resultado: 5 pruebas, 0 omitidas, 0 fallos

[PASA ] criterio 3 - el documento publicado solo contiene material publico
[PASA ] paso 1 - el identificador did web sale del dominio configurado
[PASA ] criterio 4 - un id desajustado se rechaza
[PASA ] criterio 1 y 2 - se sirve en la URL calculada, con la clave publica correcta y hash igual
[PASA ] no se publica por defecto ningun DID de ciudadano - solo namespaces reservados
```

Comando	Qué hace	Qué se busca
TEST_DB_URL=... ./gradlew :vdr-service:test --tests '*Erso005*' --rerun-tasks -q	Ejecuta solo las pruebas de esta ERSO; TEST_DB_URL indica la base; --rerun-tasks fuerza a repetirlas	Que pasen
el script de Python	Lee el reporte y lista cada prueba con PASA / FALLA	Ver cuál prueba demuestra qué

 **Lectura.** 5 pruebas, 0 omitidas, 0 fallos:

Prueba	Respalda
paso 1 - el identificador did web sale del dominio configurado	Paso 1 / laboratorio A
criterio 1 y 2 - se sirve en la URL calculada, con la clave publica correcta y hash igual	Criterios 1 y 2 / laboratorios C y D
criterio 3 - el documento publicado solo contiene material publico	Criterio 3 / laboratorio E
criterio 4 - un id desajustado se rechaza	Criterio 4 / laboratorio F (a)
no se publica por defecto ningun DID de ciudadano - solo namespaces reservados	Condición 3 / criterio 4

Preguntas (con respuesta)

- ¿Por qué una sola prueba cubre los criterios 1 y 2? Porque comparten el mismo procedimiento: publicar una vez y comprobar a la vez la URL, la clave y el hash.
- El criterio 4 menciona "regresión con la extensión desactivada". ¿Dónde está esa prueba? En las del criterio 3 de la ERSO 004 (criterio 3 - ...), que cubren el camino base apagado;

esta ERSO las reutiliza en lugar de repetirlas.

22.2 El recorrido de extremo a extremo (extracto del registro)

El script `scripts/e2e.sh` recorre **todo el sistema real** (TLS, certificados, contenedores) y deja un registro. Este es el extracto del bloque de esta ERSO, del último recorrido (`evidencias/e2e-20260929-155349.log`):

```
ERSo 2026-005 · Publicación del DID Document bajo did:web (laboratorio)

did:web:civica-desarrollo.avance.org.co:lab:laboratorio
https://civica-desarrollo.avance.org.co/lab/laboratorio/did.json
Clave P-256 generada. Multikey público: zDnaengL8nGYTWXuzsxDp3oiyL2Gcyefr1GDjXswqajMfHS2
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=8efc830b-4c36-497d-a186-6f73d6aa7885 audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=CREATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:lab:laboratorio#key-1
docHash=sha256:ddbff673d46c29587efa48f5dbdafda2e6a8d8bf120112e23566896c91c...
[4/4] escritura .... 201 If-Match=0 Idempotency-Key=9cacc073-7be4-41d5-984a-5124003f2b9c
{
  "operationId": "18b4c413-f059-49a7-a94c-b7833ee51925",
  "did": "did:web:civica-desarrollo.avance.org.co:lab:laboratorio",
  "purpose": "CREATE",
  "status": "CONFIRMED",
  "version": 1,
  "hash": "sha256:ddbff673d46c29587efa48f5dbdafda2e6a8d8bf120112e23566896c91c517b8",
  "publicUrl": "https://civica-desarrollo.avance.org.co/lab/laboratorio/did.json"
}
✓ PASS publicación aceptada (201, CONFIRMED)
Documento servido por HTTPS:
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/multikey/v1"
  ],
  "assertionMethod": [
    "did:web:civica-desarrollo.avance.org.co:lab:laboratorio#key-1"
  ],
  "authentication": [
    "did:web:civica-desarrollo.avance.org.co:lab:laboratorio#key-1"
  ],
  "id": "did:web:civica-desarrollo.avance.org.co:lab:laboratorio",
  "verificationMethod": [
    {
      "controller": "did:web:civica-desarrollo.avance.org.co:lab:laboratorio",
      "id": "did:web:civica-desarrollo.avance.org.co:lab:laboratorio#key-1",
      "publicKeyMultibase": "zDnaengL8nGYTWXuzsxDp3oiyL2Gcyefr1GDjXswqajMfHS2",
      "type": "Multikey"
    }
  ]
}
✓ PASS C1: se sirve por HTTPS en la URL calculada (200 application/did+json)
✓ PASS C2: hash del contenido servido == hash devuelto por el registro
✓ PASS C2: contiene la clave pública correcta (Multikey del keygen)
✓ PASS C3: sin claves privadas ni datos civiles en el documento

"error": "INVALID_DOCUMENT",
"message": "El documento no cumple el perfil de publicación",
"details": [
  "ID_MISMATCH: El id 'did:web:civica-desarrollo.avance.org.co:lab:otro' no coincide con el DID
esperado 'did:web:civica-desarrollo.avance.org.co:lab:laboratorio'",
  "INVALID_VERIFICATION_METHOD: El id 'did:web:civica-
desarrollo.avance.org.co:lab:laboratorio#key-1' debe ser una URL DID con fragmento del propio
```

```
documento",
  "UNAUTHORIZED_CONTROLLER: did:web:civica-desarrollo.avance.org.co:lab:laboratorio#key-1:
controlador no autorizado (did:web:civica-desarrollo.avance.org.co:lab...
]
}
✓ PASS C4: id desajustado se rechaza (422 INVALID_DOCUMENT)

[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío FALLÓ: HTTP 412 {"error":"PRECONDITION_FAILED","message":"No se emite desafío:
precondiciones incumplidas","details":["NAMESPACE_NOT_RESERVED"]}
✓ PASS un DID de ciudadano no se puede crear (412 NAMESPACE_NOT_RESERVED)
```

✓ **Lectura.** Cada línea ✓ PASS es una comprobación sobre el sistema real: la publicación se acepta y confirma, el documento se sirve por HTTPS con el tipo correcto, el hash servido coincide con el del registro, la clave publicada es la del keygen, no hay claves privadas ni datos civiles, un id ajeno se rechaza, y un DID de ciudadano no se puede crear. El mensaje [2/4] desafío FALLÓ: HTTP 412 ... es **el resultado esperado** de la prueba negativa, no un error del recorrido.

PARTE V — PREGUNTAS Y RESPUESTAS

Esta parte reúne **todas las preguntas de comprensión** de la ERSO 005, con su respuesta y el lugar de este informe donde se demuestra. Las predicciones de cada laboratorio están respondidas dentro de su sección; aquí se recopilan para consulta rápida.

23. Preguntas sobre los conceptos

#	Pregunta	Respuesta	Dónde se ve
1	¿Qué es un DID Document y qué contiene?	Un archivo JSON público que dice cuáles son las claves públicas de un DID y para qué sirve cada una. Contiene <code>@context</code> , <code>id</code> , <code>verificationMethod</code> (las claves), <code>authentication</code> , <code>assertionMethod</code> y, opcionalmente, <code>service</code>	§6, §17.2
2	¿Qué NO debe contener jamás?	Claves privadas, credenciales personales ni datos civiles	§20
3	¿Cómo se convierte un <code>did:web</code> en una dirección?	Los <code>:</code> después del dominio pasan a <code>/</code> , se añade <code>/did.json</code> y se usa <code>https://</code> . Sin segmentos, la ruta es <code>/.well-known/did.json</code>	§13.1, §16.1
4	¿Cómo se escribe un puerto en un DID?	Como <code>%3A</code> (por ejemplo <code>dominio%3A8443</code>), para no confundirlo con un separador; en la URL vuelve a ser <code>:8443</code>	§16.1
5	¿Por qué se rechazan <code>..</code> y los espacios en una ruta?	Porque <code>..</code> "sube" una carpeta y podría salirse del espacio publicado, y los espacios son ambiguos. Cada segmento solo admite letras, números y <code>. _ -</code>	§16.2
6	¿Qué es Multikey y por qué <code>zDn...</code> ?	Una forma estándar de escribir una clave pública como texto. La <code>z</code> indica base58btc y, al decodificar, los dos primeros bytes (<code>0x8024</code>) dicen que es una clave P-256; el resto es el punto comprimido de 33 bytes. Por eso toda clave P-256 en Multikey empieza por <code>zDn</code>	§17.3
7	¿Por qué <code>authentication</code> y <code>assertionMethod</code> apuntan a la misma clave?	Porque aquí una sola clave se usa para identificarse y para firmar afirmaciones; el documento lo declara apuntando a <code>...#key-1</code> desde ambas listas	§17.2
8	¿Qué es la "lista blanca"? ¿Por qué es mejor que una "lista negra"?	Aceptar solo lo conocido y rechazar el resto. Una lista negra exige prever todo lo peligroso y falla ante lo imprevisto; la blanca falla hacia el lado seguro	§13.2, §20.2
9	¿Por qué se publica el documento en forma "canónica"?	Para que lo que se hashea , lo que se firma y lo que se sirve sean los mismos bytes; así comparar hashes es una comprobación exacta	§13.3, §18.3
10	¿Por qué la lectura pública no necesita autenticación?	Porque cualquiera debe poder consultar una clave pública para verificar firmas; no hay nada secreto en el documento	§18

#	Pregunta	Respuesta	Dónde se ve
11	¿Qué significa <code>Cache-Control: no-cache</code> ?	"Antes de reutilizar una copia, pregunta si sigue vigente": evita servir una clave rotada o un DID desactivado desde una copia vieja	§18.2
12	¿Qué riesgos tiene <code>did:web</code> que no tiene un método basado en cadena de bloques?	Depende del DNS , del HTTPS y del operador del dominio : quien controla el dominio controla lo que se publica	§7 (condición 1), §27
13	¿Por qué no se publica por defecto un DID permanente de cada ciudadano?	Porque un identificador público y persistente permite correlacionar a una persona entre contextos distintos	§7 (condición 3), §21.2
14	¿Qué es ION y por qué no se integra?	Un método DID cuyos registros se anclan en una cadena de bloques. La ERSO lo excluye para mantener el piloto simple	§7 (condición 1)

24. Preguntas por laboratorio y por criterio

Paso 1 — Generar el identificador (laboratorio A)

Pregunta	Respuesta	Dónde
¿Cuál es la URL del DID <code>...:lab:<nombre></code> ?	<code>https://<dominio>/lab/<nombre>/did.json</code>	§16.1
¿Cómo se escribe el puerto en el DID y en la URL?	<code>%3A8443</code> en el DID; <code>:8443</code> en la URL	§16.1
¿Qué pasa con las mayúsculas del dominio?	Se normaliza a minúsculas	§16.1
¿Qué hace el sistema con <code>..</code> o espacios?	Los rechaza (<code>InvalidDidException</code>); el servidor repite la validación (<code>DID_INVALID</code>)	§16.2, §21.2

Paso 2 — Construir el documento (laboratorio B)

Pregunta	Respuesta	Dónde
¿Qué campos tiene el archivo de claves?	<code>privateKeyPkcs8</code> (privada) y <code>publicKeyMultibase</code> (pública)	§17.1
¿Qué propiedades tiene el documento?	<code>@context</code> , <code>id</code> , <code>verificationMethod</code> , <code>authentication</code> , <code>assertionMethod</code>	§17.2
¿Aparece alguna palabra de clave privada?	No: el conteo es <code>0</code>	§17.2
¿Qué contiene <code>zDn...</code> por dentro?	Prefijo <code>8024</code> (P-256) + punto comprimido (<code>02/03</code> + 32 bytes)	§17.3

Criterio 1 — Se sirve por HTTPS en la URL calculada (laboratorio C)

Pregunta	Respuesta	Dónde
La publicación, ¿qué código y qué estado?	201 , CONFIRMED , versión 1	§18.1
La lectura pública, ¿código y tipo?	200 , application/did+json	§18.2
¿Quién agrega ETag , X-Content-Hash y nosniff ?	La aplicación agrega ETag , Cache-Control y X-Content-Hash ; nginx agrega nosniff	§18.2
¿Qué código dan un POST , index.html y un DID inexistente?	403 , 404 y 404	§18.4
¿Por qué se usa --resolve si el DID no lleva puerto?	Porque en esta máquina el puerto 443 del servidor está publicado como 8443 y el nombre del dominio aún no apunta a ella en el DNS real	§12.1

Criterio 2 — Clave pública correcta y hash (laboratorio D)

Pregunta	Respuesta	Dónde
Los tres hashes (servido, declarado, registrado), ¿son iguales?	Sí, idénticos	§19.1
La clave generada y la publicada, ¿son iguales?	Sí	§19.2
¿Cómo se prueba que la clave publicada sirve , y no solo que es igual como texto?	Firmando con la privada y verificando con solo lo publicado : el consumidor responde VALIDA	§19.3
Si se altera un carácter, ¿qué pasa con el hash?	Cambia por completo (efecto avalancha)	§19.4
¿Qué demuestra --purpose assertionMethod ?	Que la clave está autorizada para firmar afirmaciones	§19.3

Criterio 3 — Sin material privado ni datos civiles (laboratorio E)

Pregunta	Respuesta	Dónde
¿Qué propiedades tiene el documento publicado?	Solo las cinco autorizadas	§20.1
¿Cuántas coincidencias hay con palabras de clave privada? ¿Y con el valor real de la clave privada?	0 y 0	§20.1
¿Por qué buscar el valor de la clave privada y no solo palabras?	Porque una clave podría colarse con un nombre de campo inocente; buscar el valor real es la comprobación más fuerte	§20.1
De los cinco documentos con material prohibido, ¿se publica alguno?	Ninguno: todos 422 INVALID_DOCUMENT	§20.2
¿Por qué credentialSubject falla dos veces?	Por ser dato civil (CIVIL_DATA) y por no estar en la lista blanca (UNKNOWN_PROPERTY): doble red	§20.2

Pregunta	Respuesta	Dónde
¿Queda algo guardado tras los rechazos?	No: 0 en la base y 404 público; el rechazo ocurre antes de escribir	§20.3

Criterio 4 — `id` desajustado y camino base apagado (laboratorio F)

Pregunta	Respuesta	Dónde
¿Qué responde el registro a un documento cuyo <code>id</code> es de otro DID?	422 <code>INVALID_DOCUMENT</code> con <code>ID_MISMATCH</code> ; no queda nada guardado	§21.1
¿Por qué importa el <code>id</code> ?	Evita publicar, en la ruta de A, un documento que dice ser B	§21
DID de ciudadano / dominio ajeno / ruta <code>..</code> : ¿qué pasa y cuándo?	412 (<code>NAMESPACE_NOT_RESERVED</code> , <code>DID_DOMAIN_MISMATCH</code> , <code>DID_INVALID</code>) antes de entregar el desafío	§21.2
¿Cuántos DID de ciudadanos hay publicados?	0	§21.2
Con la extensión apagada, ¿qué responden <code>/health</code> , <code>/base/ping</code> y el <code>did.json</code> ?	UP/disabled, pong y 404	§21.3

Pruebas automáticas

Pregunta	Respuesta	Dónde
¿Cuántas pruebas hay para esta ERSO y cuántas pasan?	5; las 5 pasan	§22.1
¿Por qué una sola prueba cubre los criterios 1 y 2?	Comparten el procedimiento: publicar una vez y comprobar URL, clave y hash	§22.1

PARTE VI — CÓMO RESPONDER A CADA CRITERIO

Un criterio de aceptación se responde con tres elementos: **afirmación + prueba + dónde verla**. A continuación, la redacción modelo.

25. Redacción modelo de las cuatro respuestas

Criterio 1 — El documento se sirve por HTTPS en la URL calculada

Afirmación. El DID Document publicado se sirve por HTTPS en la dirección que se deduce del propio DID, con el tipo de contenido correcto, y solo se puede leer: la puerta pública no admite escritura ni sirve otras rutas.

Prueba. (1) La publicación devuelve 201 CONFIRMED con una `publicUrl` igual a la dirección calculada. (2) Un GET por HTTPS a esa dirección devuelve 200 OK con Content-Type: `application/did+json`, ETag, X-Content-Hash y nosniff. (3) Un POST devuelve 403, otra ruta 404 y un DID inexistente 404. (4) El consumidor de DID resuelve el DID usando solo esa dirección.

Dónde verla. Laboratorio C (§18); prueba automática `Erso005DidWebTest > criterio 1 y 2 - se sirve en la URL calculada...` (§22); extracto del recorrido de extremo a extremo (§22.2).

Criterio 2 — Contiene la clave pública correcta

Afirmación. La clave publicada en el documento es la clave pública generada por la entidad, y el contenido servido es exactamente el que se escribió.

Prueba. (1) Tres huellas idénticas: la del contenido servido, la declarada en X-Content-Hash y la registrada en la última versión. (2) El `publicKeyMultibase` publicado es idéntico al del archivo de claves generado. (3) Una firma hecha con la clave privada se valida (VALIDA) usando solo el documento publicado. (4) Alterar un carácter cambia el hash.

Dónde verla. Laboratorio D (§19); prueba automática del mismo nombre que la anterior (§22).

Criterio 3 — No expone claves privadas ni datos civiles

Afirmación. El documento publicado solo contiene material público de verificación, y el sistema impide publicar claves privadas, datos civiles, propiedades no autorizadas o servicios no seguros.

Prueba. (1) El documento publicado tiene solo cinco propiedades autorizadas. (2) Cero coincidencias con palabras de clave privada y con datos civiles, y cero con el valor real de la clave privada. (3) Cinco documentos con material prohibido se rechazan con 422

INVALID_DOCUMENT y los códigos PRIVATE_KEY_MATERIAL, CIVIL_DATA, UNKNOWN_PROPERTY e INVALID_SERVICE. (4) Tras los rechazos hay 0 registros y la lectura pública da 404.

Dónde verla. Laboratorio E (§20); prueba automática criterio 3 - el documento publicado solo contiene material publico (§22).

Criterio 4 — id desajustado rechazado y camino base con la extensión apagada

Afirmación. Un documento cuyo id no coincide con el DID donde se publica es rechazado antes de guardarse, los DID no autorizados no pueden crearse, y con la extensión desactivada el camino base opera sin cambios.

Prueba. (1) Documento con id ajeno → 422 ID_MISMATCH, 0 registros y 404 público. (2) DID de ciudadano, dominio ajeno y ruta .. → 412 antes del desafío (NAMESPACE_NOT_RESERVED, DID_DOMAIN_MISMATCH, DID_INVALID). (3) 0 DID de ciudadanos publicados. (4) Con la extensión apagada: /health UP con disabled, /base/ping pong, did.json 404.

Dónde verla. Laboratorio F (§21); pruebas automáticas criterio 4 - un id desajustado se rechaza y no se publica por defecto ningún DID de ciudadano... (§22); regresión completa en el informe de la ERSO 004, criterio 3.

26. Lista de comprobación para quien acepta

 **La prueba funcional está fijada para el viernes 2 de octubre de 2026, 7:30 a. m.**
Esta lista sirve para prepararla.

✓	Qué comprobar	Qué ejecutar	Resultado esperado
☐	Entorno vivo y espacio lab/* reservado	Comandos de §12	UP/enabled; lab/* y lab/laboratorio de lab-operator
☐	Paso 1 · generar el did:web	Laboratorio A (§16)	DID y URL correctos; puerto %3A; minúsculas; .. y espacios rechazados
☐	Paso 2 · construir el documento	Laboratorio B (§17)	5 propiedades; Multikey; 0 menciones de clave privada; prefijo 8024
☐	Criterio 1 · se sirve por HTTPS	Laboratorio C (§18)	201 CONFIRMED; 200 application/did+json; POST 403; otra ruta 404; resolución correcta
☐	Criterio 2 · clave correcta y hash	Laboratorio D (§19)	3 hashes idénticos; claves idénticas; VALIDA; hash distinto si se altera
☐	Criterio 3 · sin material privado	Laboratorio E (§20)	0 y 0; 5 intentos 422; 0 registros
☐	Criterio 4 · id desajustado + regresión	Laboratorio F (§21)	422 ID_MISMATCH; 412 ×3; 0 DID de ciudadanos; disabled / pong / 404
☐	Pruebas automáticas	Laboratorio G (§22)	5 pruebas, 0 omitidas, 0 fallos
☐	Todo en un solo comando	bash scripts/lab-005-completo.sh	Las mismas salidas que este informe

✓	Qué comprobar	Qué ejecutar	Resultado esperado
☐	Recorrido E2E (reinicia el entorno)	bash scripts/e2e.sh	RESULTADO: ... PASS · 0 FAIL

PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN

27. Lo que este informe NO demuestra, y lo que se encontró

27.1 Hallazgo: HEAD responde 404 aunque GET funciona

Mientras se preparaban las transcripciones apareció una diferencia entre lo que **dice** la configuración y lo que **hace** el sistema:

```
$ curl -s -I $PUB https://$D:8443/lab/$ID/did.json | head -3
HTTP/1.1 404 Not Found
Server: nginx
Date: Thu, 01 Oct 2026 16:32:10 GMT
```

Dónde	Qué dice o hace
nginx.conf	limit_except GET HEAD { deny all; } → permite HEAD
Server.kt	Solo define get(...); no existe una ruta head(...)
Resultado	Un HEAD pasa nginx pero la aplicación responde 404

- **Qué es HEAD**. El mismo pedido que GET, pero que devuelve **solo los encabezados** (sirve para comprobar que algo existe sin descargarlo).
- **Impacto. Ningún criterio de la ERSO lo exige**: todas las comprobaciones se hacen con GET, y la especificación de did:web pide GET. Pero un cliente que sondee con HEAD creería que el documento **no existe**.
- **Recomendación**. Corregirlo antes o después de la aceptación: añadir una ruta head equivalente (o el complemento de Ktor que responde HEAD a partir de GET) y su prueba. **No se modificó el código** para no cambiar el sistema sin que el equipo lo decida.

27.2 Límites generales

Límite	Explicación
TLS de laboratorio	Los certificados los firma una autoridad propia . Para producción hace falta una autoridad reconocida y DNS del dominio
No se probó en el dominio real ni desde internet	Todo se ejecutó en una máquina local con --resolve
did:web depende de terceros	DNS, HTTPS y el operador del dominio. Es un riesgo que la ERSO reconozca, no uno que resuelva
Multikey y Base58 son implementación propia	Verificadas por ida y vuelta (20 claves aleatorias) y por el prefijo zDn; no contrastadas con vectores de terceros. Recomendado: compararlas con una biblioteca independiente antes de producción

Límite	Explicación
Discrepancia de formato de clave	El documento general del equipo ("Identidad Digital EUDI Wallet") ilustra <code>JsonWebKey2020 / publicKeyJwk</code> ; la ERSO 005 exige Multikey . Se siguió la ERSO y el sistema rechaza otros tipos de clave. Debe acordarse con el equipo
Contexto del documento	Se usa <code>https://www.w3.org/ns/did/v1</code> ; el validador acepta también <code>.../v1.1</code> . Confirmar cuál corresponde a la versión de DIDs vigente
Reserva <code>lab/*</code>	Se añadió para poder publicar un DID nuevo en cada ejecución. Cada ejecución deja un DID de prueba en el registro (un DID publicado solo puede desactivarse, no borrarse)
Coherencia con la ERSO 003	La condición 3 de esta ERSO se cumple para el laboratorio; la ERSO 003 abre un camino distinto y explícito (el Wallet Backend publica <code>titulares/<huella></code>). El riesgo de correlación de ese camino está en el informe de la 003
Alcance	La aceptación se limita al laboratorio de pruebas de identidad, como indica la condición 2
Firmas y plantilla de pruebas	Corresponden a las personas responsables (Rocío Villamizar, Geovani Rincón, Luis González); este informe aporta evidencias técnicas

28. Si algo no sale como en el informe

Síntoma	Causa más probable	Qué hacer
<code>NAMESPACE_NOT_RESERVED</code> al publicar en <code>lab/...</code>	No está reservado <code>lab/*</code> para <code>lab-operator</code>	Revisa <code>VDR_CLIENTS</code> en <code>deploy/.env</code> y <code>scripts/gen-env.sh</code> ; reinicia el proyecto (§12.5)
<code>NAMESPACE_NOT_OWNED</code>	Estás publicando en un espacio de otra entidad	Usa rutas <code>lab/...</code> con <code>lab-operator</code>
<code>ALREADY_EXISTS</code> (409)	Ese DID ya existe	Usa un nombre nuevo (<code>ID=prueba-005-\$(date +%s)</code>)
<code>curl: (60)</code> o <code>HTTP 000</code>	<code>curl</code> no conoce la CA, o falta <code>--resolve</code>	Usa <code>\$PUB</code> ; el DNS real aún no apunta a la máquina
Variables vacías (<code>\$D</code> , <code>\$ID</code> , <code>\$DID</code> ...)	Abriste un terminal nuevo	Repite §12.1 y §15; los laboratorios forman una cadena
<code>jq: error al leer c005-clave.json</code>	No existe: no ejecutaste el laboratorio B	Ejecuta el laboratorio B antes de los demás
<code>HEAD</code> devuelve 404	Hallazgo conocido (§27.1)	No afecta a los criterios; usa <code>GET</code>
Al pegar comandos largos se corrompen caracteres	La terminal pierde texto al pegar	Ejecuta <code>bash scripts/lab-005-completo.sh</code>
<code>permission denied</code> en un script	Falta permiso de ejecución	Ejecuta con <code>bash</code> delante

29. Operación diaria

Quiero...	Comando (desde <code>deploy/</code>)	Nota
Levantar el proyecto	<code>docker compose up -d</code>	
Ver el estado	<code>docker compose ps</code>	
Ver el registro de la aplicación	<code>docker compose logs --tail=50 vdr</code>	
Detenerlo conservando los datos	<code>docker compose down</code>	Los datos viven en un volumen aparte
 Borrar también los datos	<code>docker compose down -v</code>	Destructivo
Ejecutar todos los laboratorios	<code>bash ../scripts/lab-005-completo.sh</code> (o desde la raíz: <code>bash scripts/lab-005-completo.sh</code>)	Deja un DID de prueba nuevo
Regenerar el PDF de este informe	<code>bash scripts/generar-pdf.sh informes/INFORME-FINAL-ERSo-2026-005.md</code>	Requiere <code>pandoc</code> y Chrome

ANEXOS

Anexo A — Los scripts

Script	Qué hace	¿Modifica datos?
scripts/lab-005-completo.sh	Ejecuta, en una sola sesión, los laboratorios A a G con el mismo contenido que este informe	Publica un DID de prueba en lab/prueba-005-...
scripts/gen-env.sh	Crea o actualiza deploy/.env (ahora reserva también lab/*)	Crea deploy/.env
scripts/e2e.sh	Recorrido completo, con el bloque "ERSo 2026-005"	 Reinicia el entorno
scripts/generar-pdf.sh	Convierte un informe en PDF	Crea un archivo

Anexo B — Autocomprobación (con respuestas)

#	Pregunta	Respuesta
1	¿Cuál es la URL de did:web:ejemplo.org:a:b ?	https://ejemplo.org/a/b/did.json
2	¿Y la de did:web:ejemplo.org ?	https://ejemplo.org/.well-known/did.json
3	¿Cómo se escribe el puerto 8443 en un DID?	did:web:ejemplo.org%3A8443:...
4	¿Qué prefijo de bytes tiene una clave P-256 en Multikey?	0x8024 (y el texto empieza por zDn)
5	¿Cuántas propiedades lleva el documento de la ERSO 005 sin servicios?	Cinco: @context , id , verificationMethod , authentication , assertionMethod
6	¿Qué código devuelve un documento con birthDate ?	422 con CIVIL_DATA y UNKNOWN_PROPERTY
7	¿Qué código devuelve un id ajeno?	422 con ID_MISMATCH
8	¿Qué código devuelve un DID de ciudadano?	412 con NAMESPACE_NOT_RESERVED , antes de entregar el desafío
9	¿Qué prueba que la clave publicada es la correcta además de ser un texto igual?	Que valida una firma hecha con la clave privada usando solo el documento publicado
10	¿Qué demuestra Cache-Control: no-cache ?	Que el documento no debe servirse desde una copia sin preguntar si sigue vigente

Anexo C — Ejercicios del método (con respuesta modelo)

Ejercicio 1. *Aplicar la receta al criterio 4.*

Campo	Respuesta
Verbo / objeto / evidencia	Se rechaza y opera / un documento con <code>id</code> ajeno, y el camino base / pruebas negativas y de regresión
Afirmación comprobable	Un documento cuyo <code>id</code> no coincide con el DID de su ruta no se guarda ni se sirve; con la extensión apagada el camino base responde
Casos negativos	<code>id</code> de otro DID; DID de ciudadano; dominio ajeno; segmento <code>..</code>
Mecanismo (capa)	El validador compara <code>id</code> con el DID esperado (aplicación); las precondiciones se verifican antes del desafío (aplicación); las rutas del registro no se registran si está apagado (arranque)
Prueba automática	<code>criterio 4 - un id desajustado se rechaza</code>
Evidencia E2E	C4: <code>id desajustado se rechaza (422 INVALID_DOCUMENT)</code>
Límites	La regresión completa se prueba en la ERSO 004; solo <code>did:web</code>

Ejercicio 2. *Propón un sexto intento de documento prohibido para el criterio 3.* Por ejemplo, añadir `"controller": "did:web:otra.entidad"` → `UNAUTHORIZED_CONTROLLER`; o un documento mayor de 64 KiB → `T00_LARGE`. *Capa que lo frena:* el validador de contenido.

Ejercicio 3. *¿Por qué usar una lista blanca y no una lista negra?* Porque una lista negra obliga a anticipar todo lo peligroso y cualquier campo imprevisto pasaría; con una blanca, lo no previsto se **rechaza por defecto**.

Anexo D — Referencias

- W3C — *Decentralized Identifiers (DIDs) v1.1* (modelo de datos del documento).
- W3C CCG — *did:web Method Specification* (la regla DID → URL).
- W3C — *Controlled Identifiers / Multikey y multibase* (formato de la clave pública).
- IETF — RFC 3986 (URI y codificación con `%`), RFC 9110 (métodos y encabezados HTTP, incluidos `ETag` y `Cache-Control`).
- Enlaces y estado de cada especificación: `docs/VERSIONES-NORMATIVAS.md` (**confirmar siempre la versión vigente**).

Fin del informe. Elaborado para la ERSO 2026-005 del proyecto Identidad Digital Soberana SSI. Las salidas de terminal se obtuvieron ejecutando los comandos reales el 1 de octubre de 2026; los valores variables (identificadores, claves, fechas) cambian en cada ejecución, no así los patrones que se explican en cada sección.